

Mobile Development :

9 : Case Study : MedBox As a Mobile App

+ BloC Architecture + Structuring Project..



Professor Imed Bouchrika

National School of Artificial Intelligence
imed.bouchrika@ensia.edu.dz

Outline :

- **BloC Design Pattern:**
 - *Cubit*
 - *Defining your State Class*
 - *BloC*
- **Case study & Structuring your Mobile App**
 - *Folders & Architectures*
 - *Custom Widgets*
 - *Localisation*
 - *Loading data*

Section 1

BloC Architecture



BloC

- **BloC Libraries**

- **Cubit**

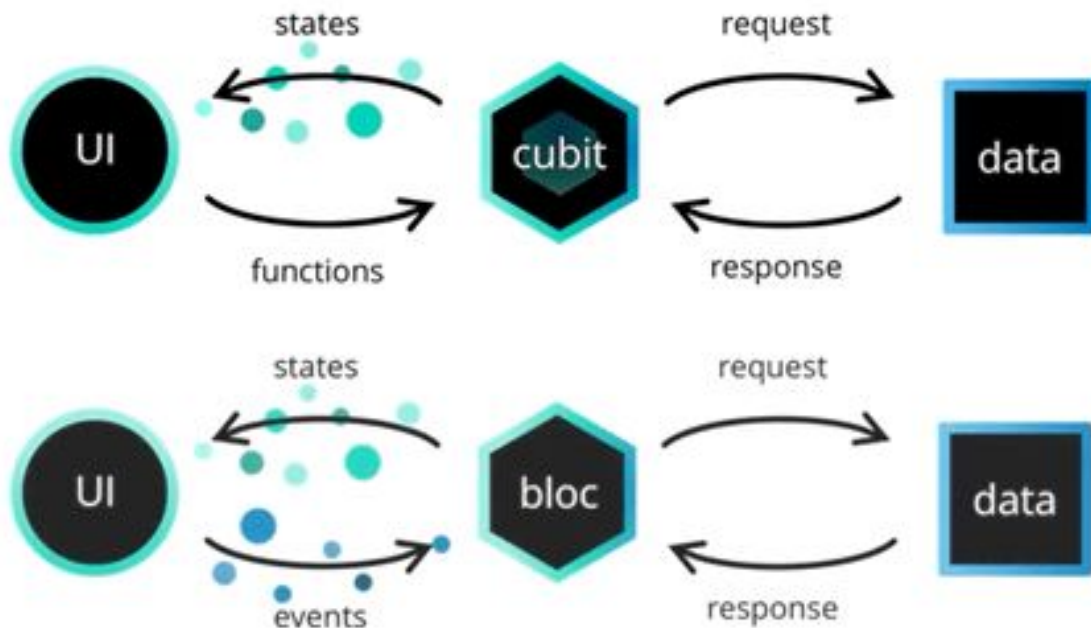
- A lightweight state management class for managing a single state.
 - Emits new states when triggered by methods.

- **Bloc:**

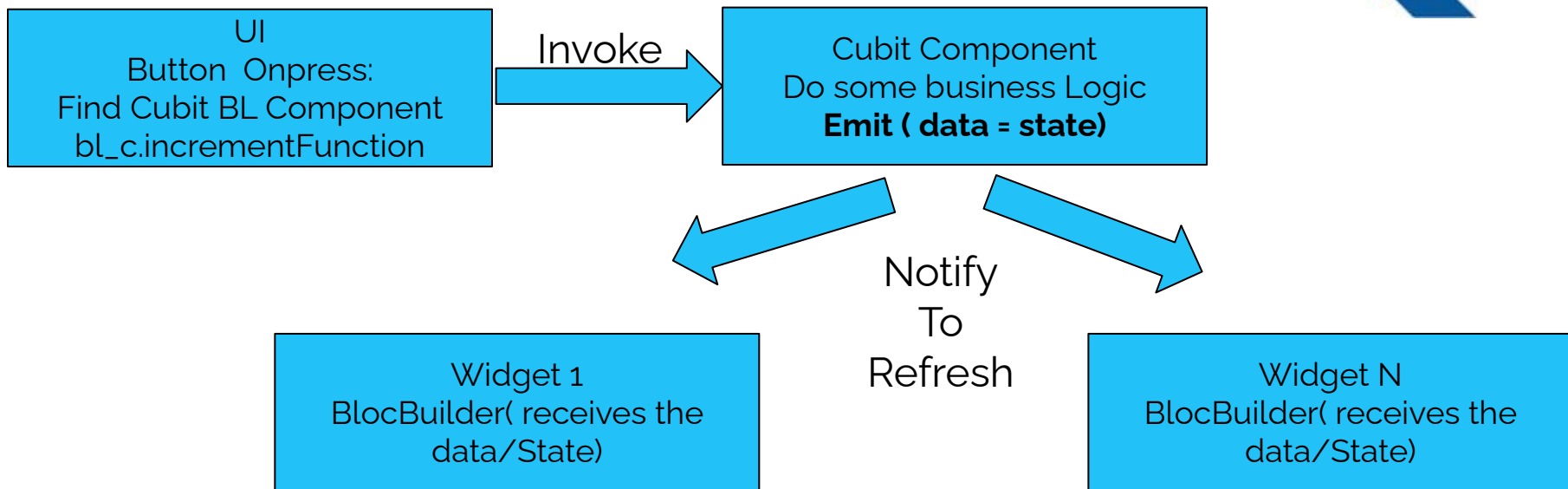
- Extends Cubit but introduces an event-based approach.
 - Reacts to incoming events and maps them to states.

BloC

- Flow of Actions



BloC/Cubit



BloC

- **Why to you must use BloC**

- **Clear Separation of Concerns:**

- Business logic is separate from UI, making code easier to maintain and test.

- **Reactiveness:**

- Widgets automatically rebuild when a new state is emitted.

- **Powerful State Management:**

- States are managed in a structured, **predictable way** with possibility to share them across widgets, screens...

- **Testability:**

- Business logic is encapsulated in Bloc and can be tested independently of UI.

BloC

- BloC Widgets

- ***BlocBuilder:***

- Like the FutureBuilder, it rebuilds a widget in response to state changes **continuously**

- ***BlocListener:***

- Reacts to state changes but does **NOT** rebuild widgets.

- ***BlocConsumer:***

- **Combines** BlocBuilder and BlocListener for both rebuilding and reacting to states.

- ***BlocProvider + MultiBlocProvider***

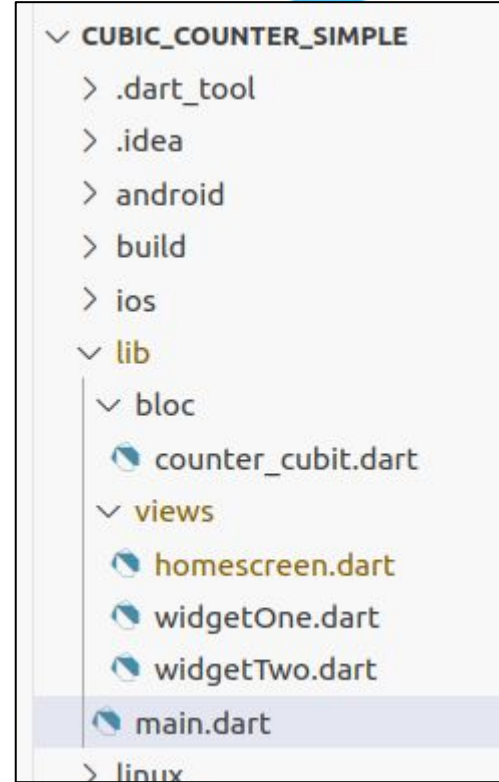
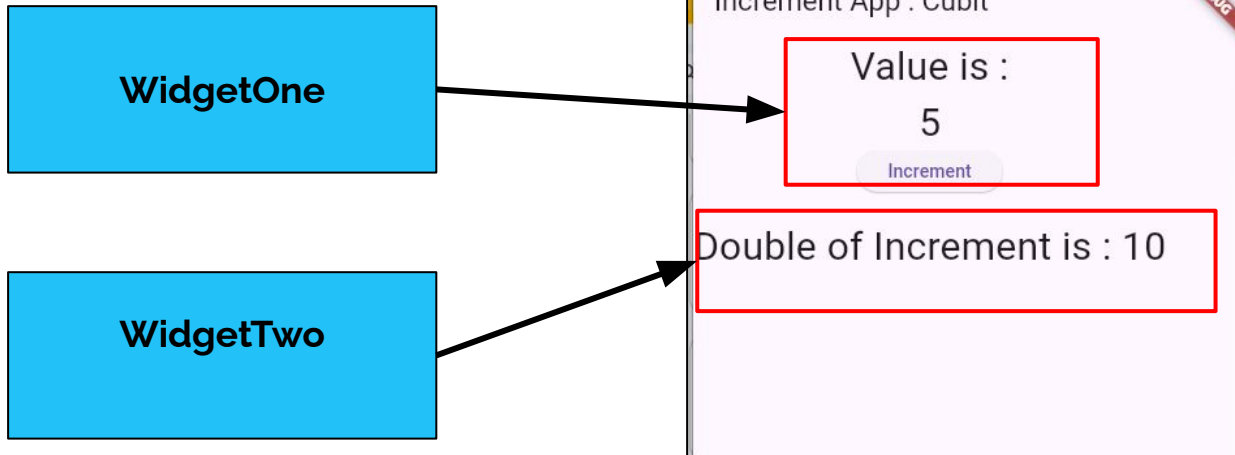
- Provides Bloc Objects to the widget tree



Integrating Cubit/BloC

BloC

- Simple Incrementing App using BloC/Cubit



main.dart

```
import 'package:cubic_counter_simple/views/homescreen.dart' ;
import 'package:flutter/material.dart' ;
import 'package:flutter_bloc/flutter_bloc.dart' ;

import 'bloc/counter_cubit.dart' ;

void main() {
  runApp(const MainApp());
}

class MainApp extends StatelessWidget {
  const MainApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MultiBlocProvider (
      providers: [
        BlocProvider(create: (_) => CounterCubit()),
      ],
      child: MaterialApp (
        home: HomeScreen(),
      ));
  }
}
```

✓ CUBIC_COUNTER_SIMPLE

> .dart_tool

> .idea

> android

> build

> ios

✓ lib

✓ bloc

• counter_cubit.dart

✓ views

• homescreen.dart

• widgetOne.dart

• widgetTwo.dart

• main.dart

> linux

01

main.dart

```
import 'package:cubic_counter_simple/views/homescreen.dart';
import 'package:flutter/material.dart';
import 'package:flutter_bloc/flutter_bloc.dart';

import 'bloc/counter_cubit.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MultiBlocProvider(
      providers: [
        BlocProvider(create: (_) => CounterCubit()),
      ],
      child: MaterialApp(
        home: HomeScreen(),
      ));
  }
}
```

**Providers : Create an instance of X or Z
and provide them to other components
or classes where it can be injected with
all its resources and dependencies
easily.
“Loose Coupling”**

main.dart

```
import 'package:cubic_counter_simple/views/homescreen.dart';
import 'package:flutter/material.dart';
import 'package:flutter_bloc/flutter_bloc.dart';

import 'bloc/counter_cubit.dart';

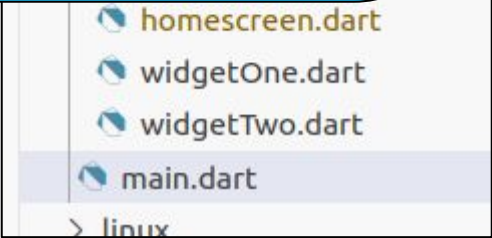
void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MultiBlocProvider(
      providers: [
        BlocProvider(create: (_) => CounterCubit()),
      ],
      child: MaterialApp(
        home: HomeScreen(),
      ));
  }
}
```

Almost the same as :

**Get.put (some object) to be located /
provided later on ..**



homescreen.dart
widgetOne.dart
widgetTwo.dart
main.dart

Homescreen.dart

```
import 'package:cubic_counter_simple/views/widgetTwo.dart';
import 'package:flutter/material.dart';
import 'package:flutter_bloc/flutter_bloc.dart';

import '../bloc/counter_cubit.dart';
import 'widgetOne.dart';

class HomeScreen extends StatelessWidget {
  const HomeScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text('Increment App : Cubit'),
      ),
      body: Column(
        children: [
          WidgetOne(),
          SizedBox(
            height: 20,
          ),
          WidgetTwo(),
        ],
      ),
    );
  }
}
```

✓ CUBIC_COUNTER_SIMPLE

> .dart_tool

> .idea

> android

> build

> ios

✓ lib

✓ bloc

🔗 counter_cubit.dart

✓ views

🔗 homescreen.dart

🔗 widgetOne.dart

🔗 widgetTwo.dart

🔗 main.dart

> linux

widgetOne.dart

```
class WidgetOne extends StatelessWidget {
  const WidgetOne({super.key});

  @override
  Widget build(BuildContext context) {
    return Column(children: [
      Text(
        'Value is :',
        style: TextStyle(fontSize: 30),
      ),
      BlocBuilder<CounterCubit, int>(
        builder: (context, count) {
          return Text(
            '$count',
            style: TextStyle(fontSize: 30),
          );
        },
      ),
      ElevatedButton(
        onPressed: () {
          context.read<CounterCubit>().increment();
        },
        child: Text('Increment')),
    ]);
  }
}
```

**Find a CounterCubit Object .. (That will
be PROVISIONED by the BlocProvider
system**

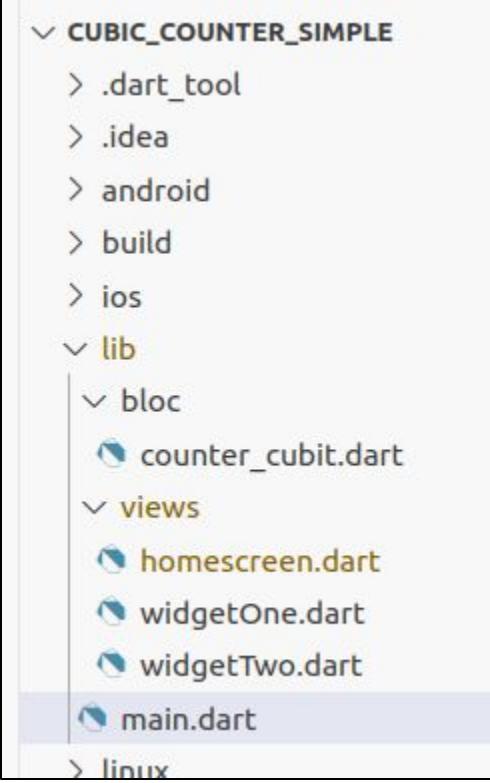
02

counter_cubit.dart

```
import 'package:bloc/bloc.dart';

class CounterCubit extends Cubit<int> {
  CounterCubit() : super(0);

  void increment() { emit(state + 1) ; }
  void decrement() { emit(state - 1) ; }
}
```



IDE file explorer showing project structure:

- ✓ CUBIC_COUNTER_SIMPLE
 - > .dart_tool
 - > .idea
 - > android
 - > build
 - > ios
 - ✓ lib
 - ✓ bloc
 - counter_cubit.dart
 - ✓ views
 - homescreen.dart
 - widgetOne.dart
 - widgetTwo.dart
 - main.dart
 - > linux

03

counter_cubit.dart

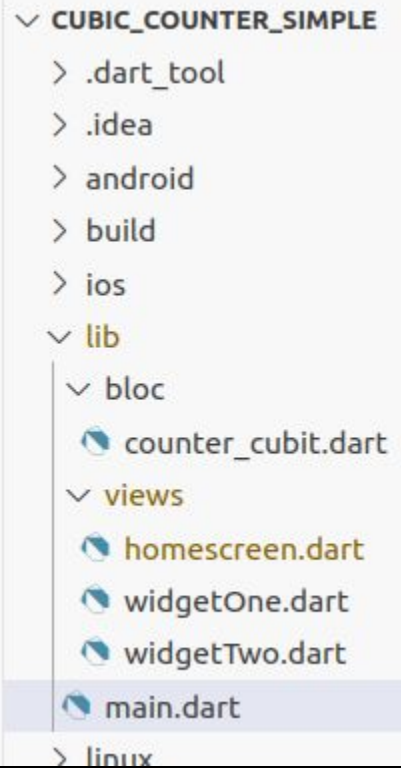
```
import 'package:bloc/bloc.dart';

class CounterCubit extends Cubit<int> {
  CounterCubit() : super(0);

  void increment() { emit(state + 1) ; }
  void decrement() { emit(state - 1) ; }
}
```

**state is a special instance variable
inherited from the Cubit**

**The state in this case is of type int
Initialized with an amount 0**



03

counter_cubit.dart

```
import 'package:bloc/bloc.dart';

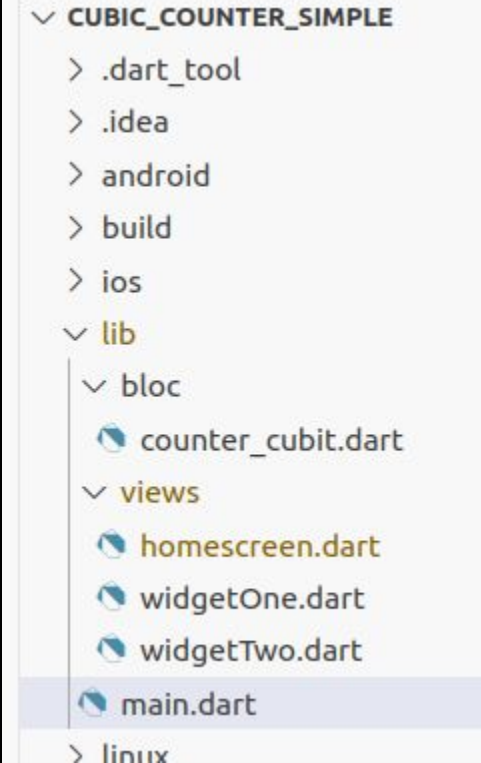
class CounterCubit extends Cubit<int> {
  CounterCubit() : super(0);

  void increment() { emit(state + 1) ; }
  void decrement() { emit(state - 1) ; }
}
```

Every time you **EMIT (SOME_DATA)**

⇒

SOME_DATA will be assigned to the instance
variable **state**
(state = SOME_DATA)



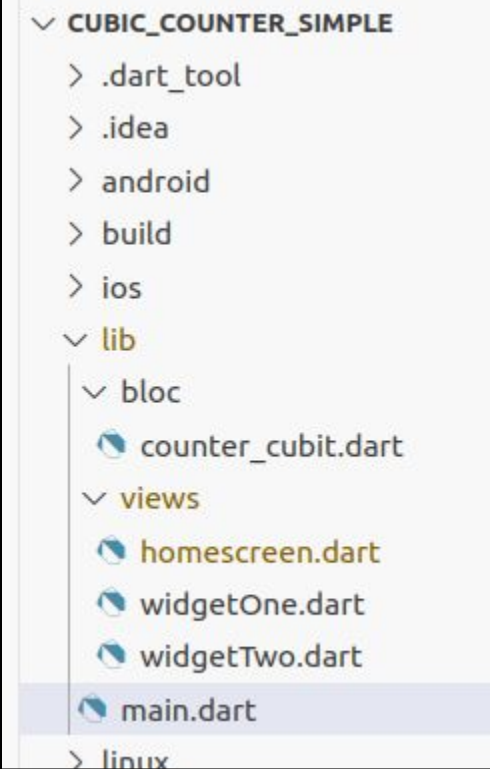
```
✓ CUBIC_COUNTER_SIMPLE
  > .dart_tool
  > .idea
  > android
  > build
  > ios
  ✓ lib
    ✓ bloc
      counter_cubit.dart
    ✓ views
      homescreen.dart
      widgetOne.dart
      widgetTwo.dart
    main.dart
  > linux
```

03

widgetOne.dart

```
class WidgetOne extends StatelessWidget {
  const WidgetOne({super.key});

  @override
  Widget build(BuildContext context) {
    return Column(children: [
      Text(
        'Value is :',
        style: TextStyle(fontSize: 30),
      ),
      BlocBuilder<CounterCubit, int>(
        builder: (context, count) {
          return Text(
            '$count',
            style: TextStyle(fontSize: 30),
          );
        },
      ),
      ElevatedButton(
        onPressed: () {
          context.read<CounterCubit>().increment();
        },
        child: Text('Increment')),
    ]);
  }
}
```



The image shows a file explorer sidebar from an IDE. It displays a project structure for 'CUBIC_COUNTER_SIMPLE'. The 'lib' directory is expanded, showing subdirectories 'bloc' and 'views'. The 'views' directory contains three files: 'homescreen.dart', 'widgetOne.dart', and 'widgetTwo.dart'. The 'main.dart' file is also visible at the bottom of the list. The 'widgetOne.dart' file is highlighted with a blue background.

- ▼ CUBIC_COUNTER_SIMPLE
 - > .dart_tool
 - > .idea
 - > android
 - > build
 - > ios
 - ▼ lib
 - ▼ bloc
 - counter_cubit.dart
 - ▼ views
 - homescreen.dart
 - widgetOne.dart
 - widgetTwo.dart
 - main.dart
 - > linux

04



Cubit States As a Class

BloC

- **Login Screen using BloC with states...**

- Exactly as we did with the FutureBuilder
 - Users puts their credentials and press Login
 - Loading circular will show for 3 seconds
 - If the username == 123, clear the stack of screens + take them to a new screen where you will be showing their username
 - Otherwise, show the login screen again with a snackBar message invalid ...

BloC

simple_login_states

Login Screen..

Username

Password

Login

simple_login_states

Login Screen..

simple_login_states

Login Screen..

Username 1111

Password

Login

Invalid username of password...

simple_login_states

Logged In

Welcome 123

Logged successfully...

main.dart

```
import 'package:flutter/material.dart';
import 'package:flutter_bloc/flutter_bloc.dart';
import 'package:simple_login_states/views/login.dart';

import 'bloc/login/login_cubit.dart';

void main() {
  runApp(const MainApp());
}

class MainApp extends StatelessWidget {
  const MainApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MultiBlocProvider(
      providers: [
        BlocProvider(create: (_) => LoginCubit()),
      ],
      child: MaterialApp(
        home: LoginScreen(),
        debugShowCheckedModeBanner: false,
      ));
  }
}
```

▼ SIMPLE_LOGIN_STATES

> .dart_tool

> .idea

> android

> build

> ios

▼ lib

▼ bloc/login

login_cubit.dart

login_state.dart

▼ views

login.dart

welcome.dart

main.dart

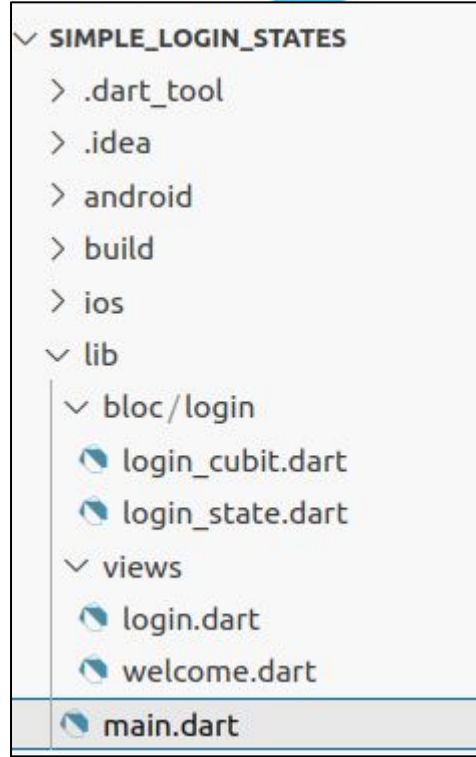
login_cubit.dart

```
import 'package:bloc/bloc.dart';
import 'package:simple_login_states/bloc/login/login_state.dart';

class LoginCubit extends Cubit<LoginState> {
  LoginCubit() : super(LoginInitial());

  Future<bool> actionProcessLogin(String username, String password) async {
    emit(LoginLoading());
    await Future.delayed(const Duration(seconds: 5)); // Simulate delay

    try {
      if (username == '123')
        emit(LoginSuccess(username)); // Emit success state
      else
        emit(LoginError('Invalid username or password...'));
    } catch (e) {
      emit(LoginError("An error occurred...")); // Emit error state
    }
    return true; // event driven architecture, has no meaning here :(
  }
}
```



▼ SIMPLE_LOGIN_STATES

- > .dart_tool
- > .idea
- > android
- > build
- > ios
- ▼ lib
 - ▼ bloc/login
 - login_cubit.dart
 - login_state.dart
 - ▼ views
 - login.dart
 - welcome.dart

main.dart

login_cubit.dart

```
import 'package:bloc/bloc.dart';
import 'package:simple_login_states/bloc/login/login_state.dart';

class LoginCubit extends Cubit<LoginState> {
  LoginCubit() : super(LoginInitial());

  Future<bool> actionProcessLogin(String username, String password) async {
    emit(LoginLoading());
    await Future.delayed(const Duration(seconds: 5)); // Simulate delay

    try {
      if (username == '123')
        emit(LoginSuccess(username)); // Emit success state
      else
        emit(LoginError('Invalid username or password...'));
    } catch (e) {
      emit(LoginError("An error occurred...")); // Emit error state
    }
    return true; // event driven architecture, has no meaning here :(
  }
}
```

▼ SIMPLE_LOGIN_STATES

- > .dart_tool
- > .idea
- > android
- > build

**This is a type of State that we
are creating**

login_state.dart

```
import 'package:equatable/equatable.dart';

abstract class LoginState extends Equatable {
  @override
  List<Object> get props => [];
}

class LoginInitial extends LoginState {
  @override
  List<Object> get props => ['loading'];
}

class LoginLoading extends LoginState {}

class LoginSuccess extends LoginState {
  final String username;
  @override
  List<Object> get props => ['logged'];

  LoginSuccess(this.username);
}

class LoginError extends LoginState {
  final String message;
  @override
  List<Object> get props => ['error'];
  LoginError(this.message);
}
```

**How to know that the current
state is different from the
newly emitted state to refresh**

?

login_state.dart

```
import 'package:equatable/equatable.dart';
```

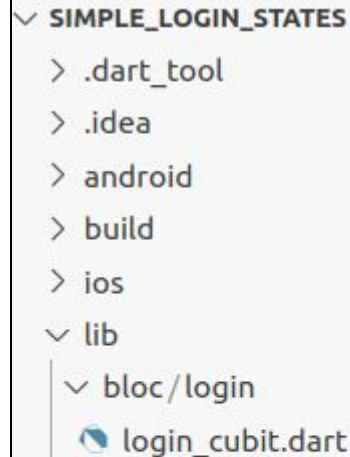
```
abstract class LoginState extends Equatable {  
  @override  
  List<Object> get props => [];  
}
```

```
class LoginInitial extends LoginState {  
  @override  
  List<Object> get props => ['loading'];  
}
```

```
class LoginLoading extends LoginState {}
```

```
class LoginSuccess extends LoginState {  
  final String username;  
  @override  
  List<Object> get props => ['logged'];  
  
  LoginSuccess(this.username);  
}
```

```
class LoginError extends LoginState {  
  final String message;  
  @override  
  List<Object> get props => ['error'];  
  LoginError(this.message);  
}
```



```
✓ SIMPLE_LOGIN_STATES  
  > .dart_tool  
  > .idea  
  > android  
  > build  
  > ios  
  ✓ lib  
    ✓ bloc/login  
      login_cubit.dart
```

flutter pub add equatable

views/login.dart

```
class LoginScreen extends StatelessWidget {
  final TextEditingController usernameController = TextEditingController();
  final TextEditingController passwordController = TextEditingController();

  Widget getLoginUI(BuildContext context) {
    return Column(
      mainAxisAlignment: MainAxisAlignment.center,
      children: [
        // Username Field
        TextField(
          controller: usernameController,
          decoration: InputDecoration(
            labelText: 'Username',
            prefixIcon: const Icon(Icons.person),
            border: OutlineInputBorder(
              borderRadius: BorderRadius.circular(8.0),
            ),
          ),
        ),
        const SizedBox(height: 16.0),
        // Password Field
        TextField(
          controller: passwordController,
          obscureText: true,
          decoration: InputDecoration(
            labelText: 'Password',
            prefixIcon: const Icon(Icons.lock),
            border: OutlineInputBorder(
              borderRadius: BorderRadius.circular(8.0),
            ),
          ),
        ),
        const SizedBox(height: 24.0),
        // Login Button

```

▼ SIMPLE_LOGIN_STATES

> .dart_tool

> .idea

> android

> build

> ios

▼ lib

▼ bloc/login

login_cubit.dart

login_state.dart

▼ views

login.dart

welcome.dart

main.dart

views/login.dart

```
class LoginScreen extends StatelessWidget {  
  final TextEditingController usernameController = TextEditingController();  
  final TextEditingController passwordController = TextEditingController();  
  
  const SizedBox(height: 24.0),  
  // Login Button  
  ElevatedButton(  
    onPressed: () {  
      String username = usernameController.text;  
      String password = passwordController.text;  
      context.read<LoginCubit>().actionProcessLogin(username, password);  
    },  
    style: ElevatedButton.styleFrom(  
      padding:  
        const EdgeInsets.symmetric(horizontal: 32.0, vertical: 12.0),  
    ),  
    child: const Text(  
      'Login',  
      style: TextStyle(fontSize: 18.0),  
    ),  
  ),  
],  
);  
}
```

▼ SIMPLE_LOGIN_STATES

- > .dart_tool
- > .idea
- > android
- > build
- > ios
- ▼ lib
 - ▼ bloc/login
 - login_cubit.dart
 - login_state.dart
 - ▼ views
 - login.dart
 - welcome.dart

main.dart

views/login.dart

```
class LoginScreen extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Login Screen..')),
      body: Center(
        child: BlocConsumer<LoginCubit, LoginState>(
          listener: (context, state) {
            if (state is LoginError) {
              // Show error snackbar
              ScaffoldMessenger.of(context).showSnackBar(
                SnackBar(
                  content: Text(state.message),
                  duration: Duration(seconds: 3),
                ),
              );
            }
            if (state is LoginSuccess) {
              Navigator.pushAndRemoveUntil(
                context,
                MaterialPageRoute(builder: (context) => WelcomeScreen()),
                (route) =>
                  false, // This condition removes all routes from the stack
              );

              // Show error snackbar
              ScaffoldMessenger.of(context).showSnackBar(
                SnackBar(
                  content: Text('Logged successfully...'),
                  duration: Duration(seconds: 3),
                ),
              );
            }
          },
        ),
      ),
    );
  }
},
```

▼ SIMPLE_LOGIN_STATES

- > .dart_tool
 - > .idea
 - > android
 - > build
 - > ios
 - ▼ lib
 - ▼ bloc/login
 - login_cubit.dart
 - login_state.dart
 - ▼ views
 - login.dart
 - welcome.dart
- main.dart

welcome.dart

```
import 'package:flutter/material.dart';
import 'package:flutter_bloc/flutter_bloc.dart';
import 'package:simple_login_states/bloc/login/login_cubit.dart';

import '../bloc/login/login_state.dart';

class WelcomeScreen extends StatelessWidget {
  const WelcomeScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text('Logged In')),
      body: BlocBuilder<LoginCubit, LoginState>(builder: (context, state) {
        if (state is LoginSuccess)
          return Text(
            'Welcome ${state.username}',
            style: TextStyle(fontSize: 30),
          );
        else
          return Text(
            'Error,,,You should not be here...',
            style: TextStyle(fontSize: 30),
          );
      }));
  }
}
```

▼ SIMPLE_LOGIN_STATES

> .dart_tool

> .idea

> android

> build

> ios

▼ lib

▼ bloc/login

login_cubit.dart

login_state.dart

▼ views

login.dart

welcome.dart

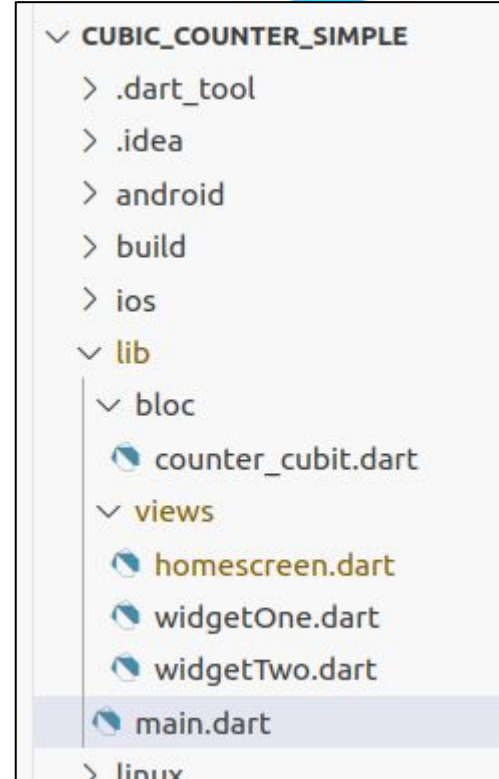
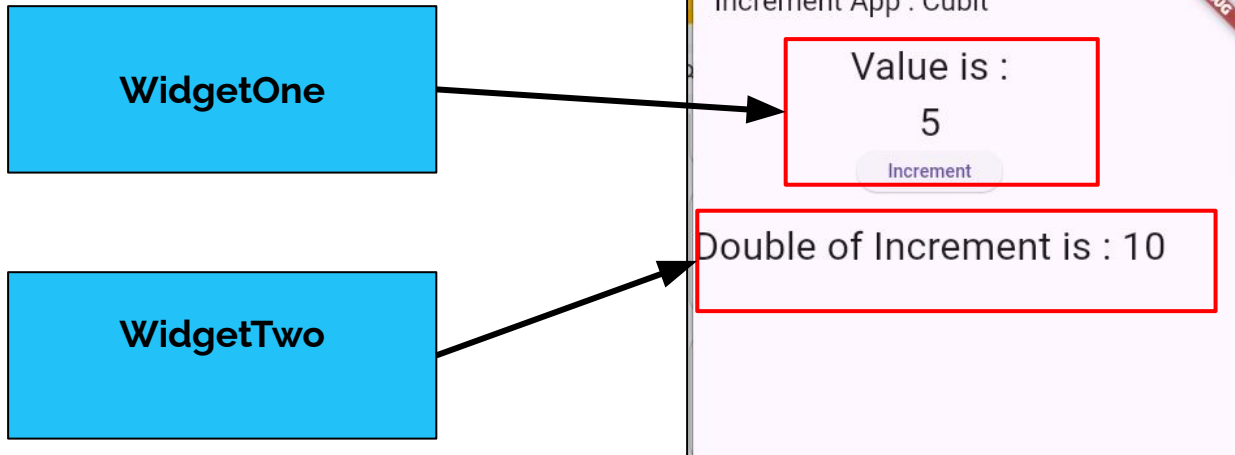
main.dart



Using Bloc /Events

BloC

- Simple Incrementing App using BloC/Cubit



main.dart

```
import 'package:cubic counter simple/views/homescreen.dart';
import 'package:flutter/material.dart';
import 'package:flutter_bloc/flutter_bloc.dart';

import 'bloc/counter_bloc.dart';

void main() {
  runApp(const MainApp());
}

class MainApp extends StatelessWidget {
  const MainApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MultiBlocProvider(
      providers: [
        BlocProvider(create: (_) => CounterBloc()),
      ],
      child: MaterialApp(
        home: HomeScreen(),
      ));
  }
}
```

Same as Cubit

▼ CUBIC_COUNTER_SIMPLE

- > .dart_tool
- > .idea
- > android

cubit.dart

green.dart

- widgetOne.dart
- widgetTwo.dart
- main.dart

> linux

01

Homescreen.dart

```
import 'package:cubic_counter_simple/views/widgetTwo.dart';
import 'package:flutter/material.dart';
import 'package:flutter_bloc/flutter_bloc.dart';

import '../bloc/counter_cubit.dart';
import 'widgetOne.dart';

class HomeScreen extends StatelessWidget {
  const HomeScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text('Increment App : Cubit'),
      ),
      body: Column(
        children: [
          WidgetOne(),
          SizedBox(
            height: 20,
          ),
          WidgetTwo(),
        ],
      ),
    );
  }
}
```

✓ CUBIC_COUNTER_SIMPLE

> .dart_tool

> .idea

> android

> build

> ios

✓ lib

✓ bloc

🔗 counter_cubit.dart

✓ views

🔗 homescreen.dart

🔗 widgetOne.dart

🔗 widgetTwo.dart

🔗 main.dart

> linux

counter_bloc.dart

```
import 'package:bloc/bloc.dart';

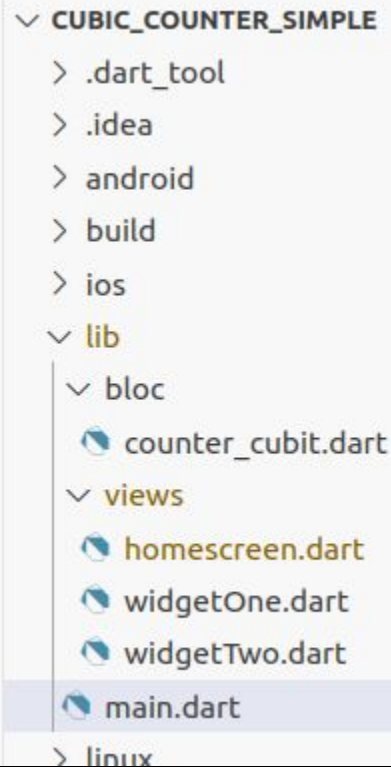
class CounterBloc extends Bloc<CounterEvent, int> {
  CounterBloc() : super(0) {
    on<IncrementEvent>((event, emit) {
      emit(state + 1);
    });

    on<DecrementEvent>((event, emit) {
      emit(state - 1);
    });
  }
}

abstract class CounterEvent {}

class IncrementEvent extends CounterEvent {}

class DecrementEvent extends CounterEvent {}
```



02

widgetOne.dart

```
import 'package:flutter/material.dart';  
import 'package:flutter_bloc/flutter_bloc.dart';
```

```
import '../bloc/counter_bloc.dart';
```

```
class WidgetOne extends StatelessWidget {  
  const WidgetOne({super.key});
```

```
@override
```

```
Widget build(BuildContext context) {
```

```
  return Column(children: [
```

```
    Text(
```

```
      'Value is :',
```

```
      style: TextStyle(fontSize: 30),
```

```
    ),
```

```
    BlocBuilder<CounterBloc, int>(  
      builder: (context, count) {
```

```
        return Text(  
          '$count',
```

```
          style: TextStyle(fontSize: 30),
```

```
        );
```

```
      ),
```

```
    ),
```

```
  ),
```

```
);
```

```
ElevatedButton(  
  onPressed: () {
```

```
    context.read<CounterBloc>().add(IncrementEvent());
```

```
  },
```

```
  child: Text('Increment'),
```

```
);
```

```
];
```

```
}
```

```
1
```

Totally different

03

widgetOne.dart

```
import 'package:flutter/material.dart';
import 'package:flutter_bloc/flutter_bloc.dart';

import '../bloc/counter_bloc.dart';

class WidgetOne extends StatelessWidget {
  const WidgetOne({super.key});

  @override
  Widget build(BuildContext context) {
    return Column(children: [
      Text(
        'Value is :',
        style: TextStyle(fontSize: 30),
      ),
      BlocBuilder<CounterBloc, int>(
        builder: (context, count) {
          return Text(
            '$count',
            style: TextStyle(fontSize: 30),
          );
        },
      ),
      ElevatedButton(
        onPressed: () {
          context.read<CounterBloc>().add(IncrementEvent());
        },
        child: Text('Increment')),
    ]);
  }
}
```

BlocBuilder ..Same as Cubit

04

Section 2

MedBox : Case Study..



Case Study for developing a Mobile app : MedBox

- **Specifications**

- Multi-profile management for family members.
- Secure document upload (images, PDFs) with categories (lab results, prescriptions).
- Categorization by date/type (e.g., diagnosis, lab, imaging)
- Export/share selected records securely (PDF)
- Medication log with dosage tracking and reminders.
- Appointment calendar with notifications.
- Health Summary Dashboard
-

Case Study for developing a Mobile app : MedBox



- **Primary Use Case**

- As a mobile app user,
 - I would like to add a family member to my App so that i later add their medical records
- As a mobile app user
 - I would like to add a medical record for a family member so that I can easily access them digitally in the future.

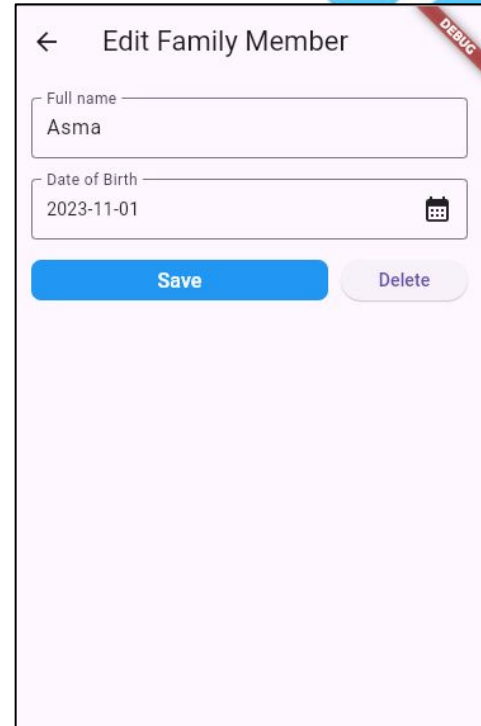
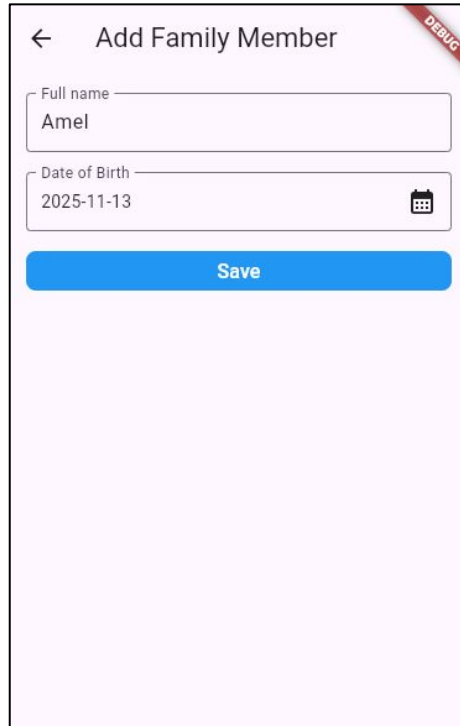
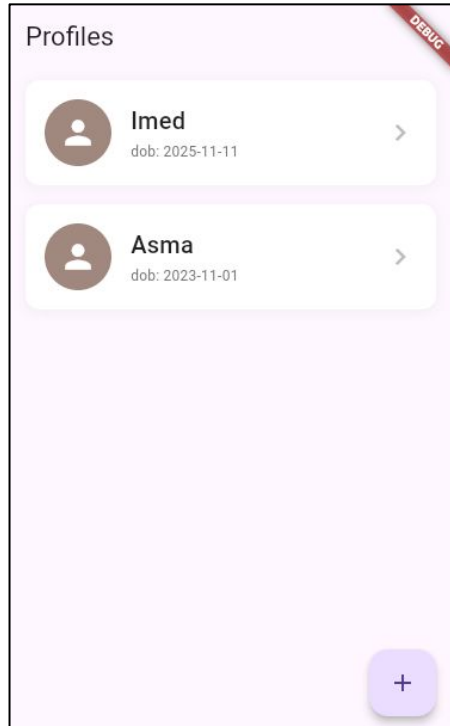
Case Study for developing a Mobile app : MedBox



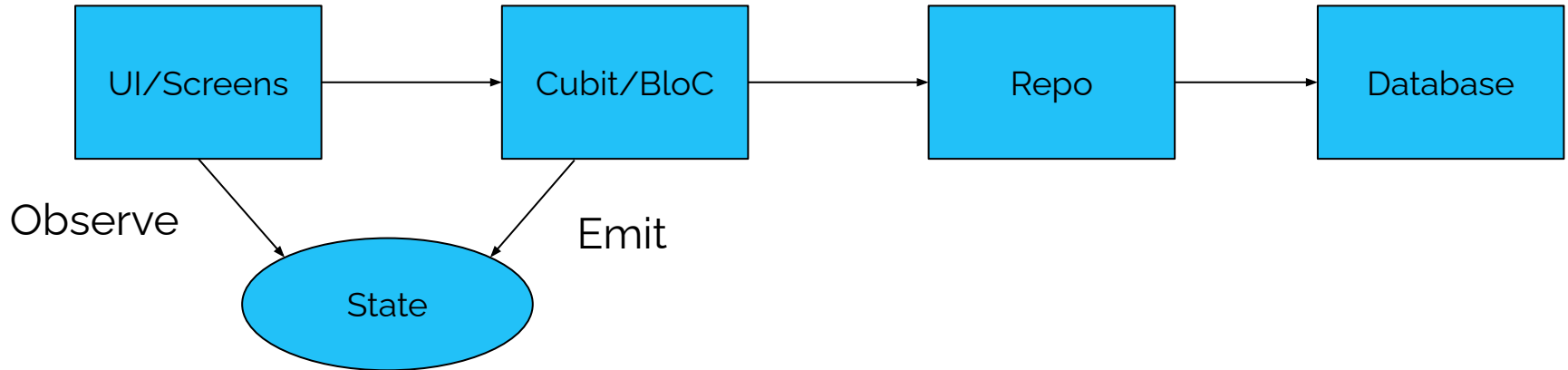
- **Primary Use Case**

- As a mobile app user,
 - I would like to add a family member to my App so that I later add their medical records
- Screens :
 - ?
 - ?

Case Study for developing a Mobile app : MedBox



Case Study for developing a Mobile app : MedBox

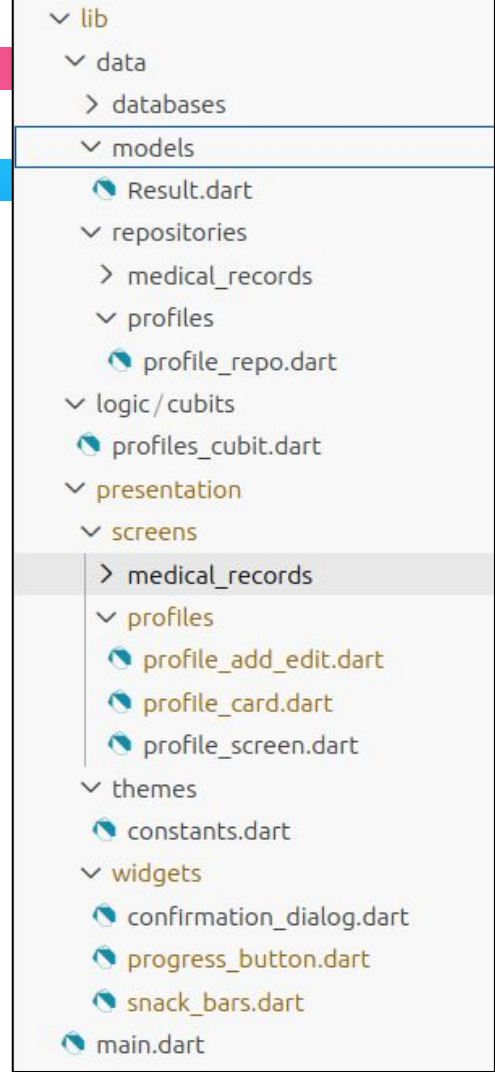




Project Structure

● Project Structure

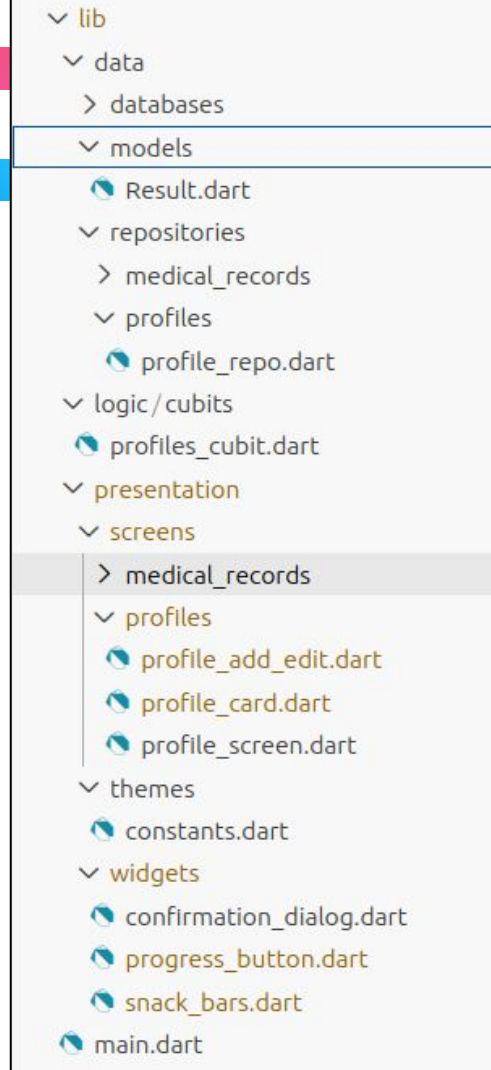
- Lib:
 - data
 - repositories
 - profiles
 - medical_records
 - models
 - databases
 - presentation
 - screens
 - profiles
 - medical_records
 - widgets
 - themes
 - logic
 - cubits
 - lon
 - **modules**
 - **authentication**
 - **screens**
 - **logic**
 - **data**
 - **upgrades**
 - **screens**
 - **logic**



Case Study for developing a Mobile app : MedBox

● Project Structure

- `dependencies:`
- `flutter:`
- `sdk: flutter`
- `flutter_bloc: ^9.1.1`
- `get_it: ^9.1.0`
- `path: ^1.9.1`
- `sqflite: ^2.4.2`
- `sqflite_common_ffi: ^2.3.6`



```

Future<bool> init_my_app() async {
  if (Platform.isLinux || Platform.isWindows) {
    sqfliteFfiInit();
    databaseFactory = databaseFactoryFfi;
  }

  final sl = GetIt.instance;
  sl.registerLazySingleton<ProfilesRepo>(() => ProfilesRepo());
  WidgetsFlutterBinding.ensureInitialized();
  return true;
}

void main() async {
  await init_my_app();
  runApp(const MainApp());
}

class MainApp extends StatelessWidget {
  const MainApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MultiBlocProvider(
      providers: [BlocProvider(create: (_) => ProfilesCubit())],
      child: const MaterialApp(home: ProfileScreen()),
    );
  }
}

```

main.dart

- ✓ lib
 - ✓ data
 - > databases
 - ✓ models
 - Result.dart
 - ✓ repositories
 - > medical_records
 - ✓ profiles
 - profile_repo.dart
 - ✓ logic/cubits
 - profiles_cubit.dart
 - ✓ presentation
 - ✓ screens
 - > medical_records
 - ✓ profiles
 - profile_add_edit.dart
 - profile_card.dart
 - profile_screen.dart
 - ✓ themes
 - constants.dart
 - ✓ widgets
 - confirmation_dialog.dart
 - progress_button.dart
 - snackBars.dart
 - main.dart


```
Future<bool> init_my_app() async {
  if (Platform.isLinux || Platform.isWindows) {
    sqfliteFfiInit();
    databaseFactory = databaseFactoryFfi;
  }

  final sl = GetIt.instance;
  sl.registerLazySingleton<ProfilesRepo>(() => ProfilesRepo());
  WidgetsFlutterBinding.ensureInitialized();
  return true;
}
```

**Initialize the Service Locator to store objects
and later can be accessed by type in a
singleton fashion**

```
const MainApp({Super.key}),

@override
Widget build(BuildContext context) {
  return MultiBlocProvider(
    providers: [BlocProvider(create: (_) => ProfilesCubit())],
    child: const MaterialApp(home: ProfileScreen()),
  );
}
```

main.dart

```

class ProfileScreen extends StatelessWidget {
  const ProfileScreen({super.key});
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text('Profiles')),
      floatingActionButton: FloatingActionButton (
        child: const Icon(Icons.add),
        onPressed: () {
          Navigator.push(context,
            MaterialPageRoute (builder: (_) => AddEditProfileScreen ()),
          );
        },
      ),
      body: Center (
        child: Container (
          child: BlocConsumer<ProfilesCubit, Map<String, dynamic>>(
            listener: (context, state) {},
            builder: (context, state) {
              if (state['state'] == 'loading') {
                return CircularProgressIndicator();
              }
              if (state['state'] == 'error') {
                return Text('Error....${state['data']['message']}');
              }
              return ListView.builder(
                itemCount: state['data'].length,
                itemBuilder: (context, index) {
                  return ProfileCard(data: state['data'][index]);
                },
              );
            },
          ),
        ),
      ),
    );
  }
}

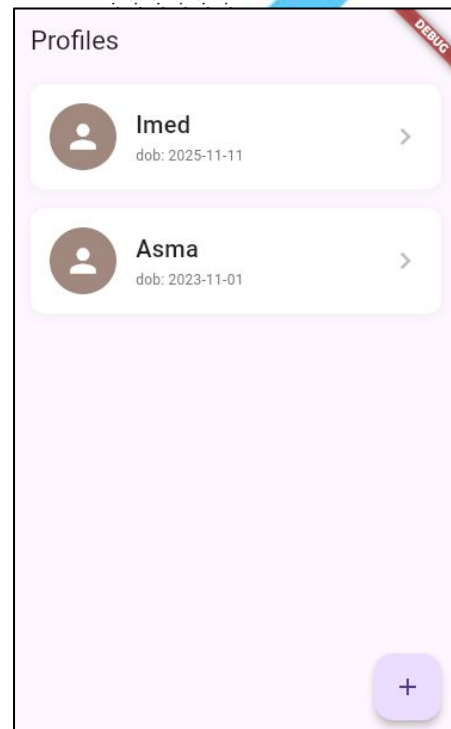
```

profile_screen.dart



profile_screen.dart

```
class ProfileScreen extends StatelessWidget {
  const ProfileScreen({super.key});
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text('Profiles')),
      floatingActionButton: FloatingActionButton (
        child: const Icon(Icons.add),
        onPressed: () {
          Navigator.push(context,
            MaterialPageRoute (builder: (_) => AddEditProfileScreen ()),
          );
        },
      ),
      body: Center (
        child: Container (
          child: BlocConsumer<ProfilesCubit, Map<String, dynamic>>(
            listener: (context, state) {},
            builder: (context, state) {
              if (state['state'] == 'loading') {
                return CircularProgressIndicator();
              }
              if (state['state'] == 'error') {
                return Text('Error....${state['data']['message']}');
              }
              return ListView.builder(
                itemCount: state['data'].length,
                itemBuilder: (context, index) {
                  return ProfileCard(data: state['data'][index]);
                },
              );
            },
          ),
        ),
      ),
    );
  }
}
```



result.dart

```
class ReturnResult {  
  final bool state;  
  final String message;
```

```
  ReturnResult({required this.state, required this.message});  
}
```

profile_cubit.dart

```
class ProfilesCubit extends Cubit<Map<String, dynamic>> {
  late final ProfilesRepo profileRepo;
  ProfilesCubit() : super({ 'data': [], 'state': 'loading', 'message': '' }, {
    profileRepo = GetIt.I<ProfilesRepo>();
    load();
  }) {
    Future<bool> load() async {
      emit({...state, 'state': 'loading', 'message': '', 'data': []});
      try {
        final records = await profileRepo.getData();
        print('Load Data..... ${records.toString()}');
        emit({...state, 'data': records, 'state': 'done', 'message': ''});
      } catch (e) {emit({...state, 'state': 'error', 'message': e.toString(), 'data': []});}
      return true;
    }

    Future<ReturnResult> insertItem(Map<String, dynamic> record) async {
      var result = ReturnResult(state: false, message: 'There is an error');
      try {
        result = await profileRepo.insertItem(record);
        await load();
      } catch (e) {emit({...state, 'error': e.toString(), 'data': []});}
      return result;
    }

    Future<ReturnResult> updateItem(Map<String, dynamic> record) async {
      var result = ReturnResult(state: false, message: 'There is an error');
      try {
        result = await profileRepo.updateItem(record);
        await load();
      } catch (e) {emit({...state, 'error': e.toString(), 'data': []});}
      return result;
    }

    Future<ReturnResult> deleteItem(int id) async {
      var result = ReturnResult(state: false, message: 'There is an error');
      try {
        result = await profileRepo.deleteItem(id);
      } catch (e) {emit({...state, 'error': e.toString(), 'data': []});}
      return result;
    }
  }
}
```

profile_cubit.dart

```
class ProfilesCubit extends Cubit<Map<String, dynamic>> {  
  late final ProfilesRepo profileRepo;  
  ProfilesCubit() : super({'data': [], 'state': 'loading', 'message': ''});  
  profileRepo = GetIt.I<ProfilesRepo>();  
  load();  
}  
Future<ReturnResult> emit({String state, String message}) async {  
  try {  
    if (state == 'loading') {  
      profileRepo = GetIt.I<ProfilesRepo>();  
      await load();  
    }  
    emit({'state': state, 'message': message});  
  } catch (e) {  
    emit({'state': 'error', 'message': e.toString(), 'data': []});  
  }  
  return ReturnResult(state: state, message: message);  
}  
Future<ReturnResult> insertItem(Map<String, dynamic> record) async {  
  var result = ReturnResult(state: false, message: 'There is an error');  
  try {  
    result = await profileRepo.insertItem(record);  
    await load();  
  } catch (e) {  
    emit({'state': 'error', 'message': e.toString(), 'data': []});  
  }  
  return result;  
}  
Future<ReturnResult> updateItem(Map<String, dynamic> record) async {  
  var result = ReturnResult(state: false, message: 'There is an error');  
  try {  
    result = await profileRepo.updateItem(record);  
    await load();  
  } catch (e) {  
    emit({'state': 'error', 'message': e.toString(), 'data': []});  
  }  
  return result;  
}  
Future<ReturnResult> deleteItem(int id) async {  
  var result = ReturnResult(state: false, message: 'There is an error');  
  try {
```

Will get an instance of the ProfileRepo from the service location

profile_repo.dart

```
class ProfilesRepo {
  List<Map<String, dynamic>> data = [
    {'id': 1, 'name': 'Imed', 'dob': '2025-11-11'},
    {'id': 2, 'name': 'Asma', 'dob': '2023-11-01'},
  ];

  Future<List<Map<String, dynamic>>> getData() async {
    await Future.delayed(Duration(seconds: 4));
    return data;
  }

  Future<ReturnResult> insertItem(Map<String, dynamic> record) async {
    await Future.delayed(Duration(seconds: 2));
    if (record['name'].length <= 2) {
      return ReturnResult(state: false, message: 'Name length should be > 2');
    }
    record['id'] = DateTime.now().millisecondsSinceEpoch;
    data.add(record);
    return ReturnResult(state: true, message: 'Record is inserted successfully');
  }

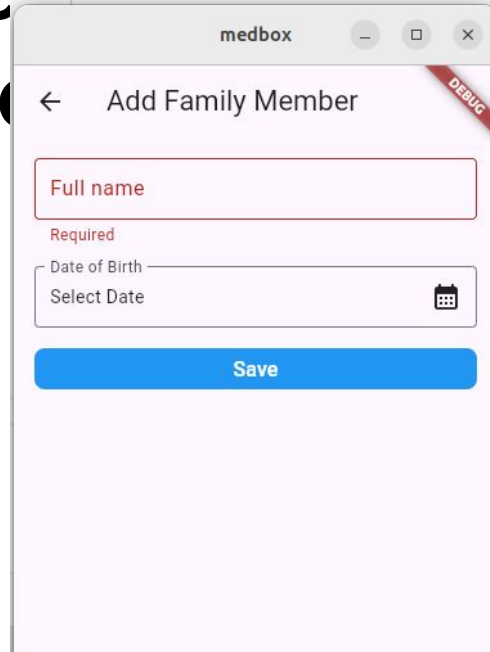
  Future<ReturnResult> updateItem(Map<String, dynamic> record) async {
    await Future.delayed(Duration(seconds: 2));
    for (int i = 0; i < data.length; i++) {
      if (data[i]['id'] == record['id']) {
        if (data[i]['name'].length <= 2) {
          return ReturnResult(
            state: false,
            message: 'Name length should be > 2',
          );
        }
        data[i] = record;
      }
    }
    return ReturnResult(state: true, message: 'Record is updated successfully');
  }

  Future<ReturnResult> deleteItem(int id) async {
```

Case Study for developing a Mobile app : Medbox

- **Validation of Data**

- On Screen
- + On Data Repo Layer
 - (Backend, database layer..)



medbox

← Add Family Member

Full name

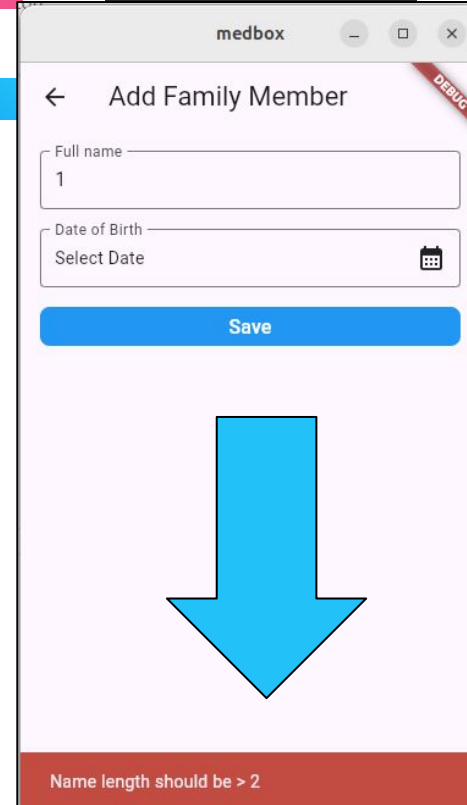
Required

Date of Birth

Select Date

Save

DEBUG



medbox

← Add Family Member

Full name

1

Date of Birth

Select Date

Save

↓

Name length should be > 2

DEBUG

Case Study for developing a Mobile app : Medbox

- Validation of Data

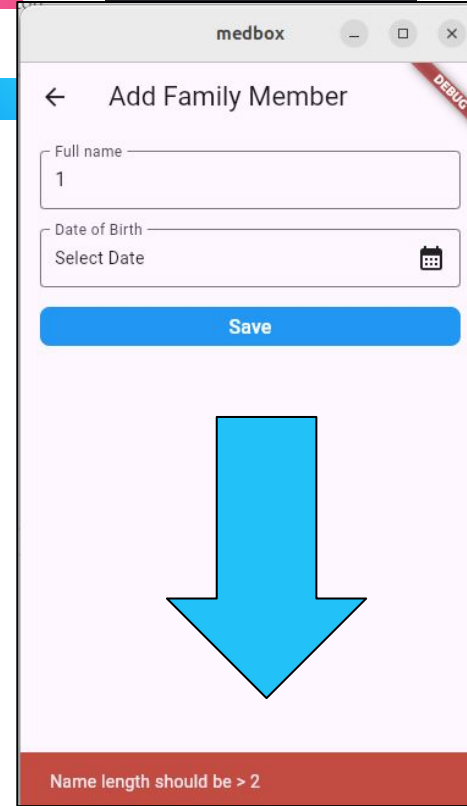
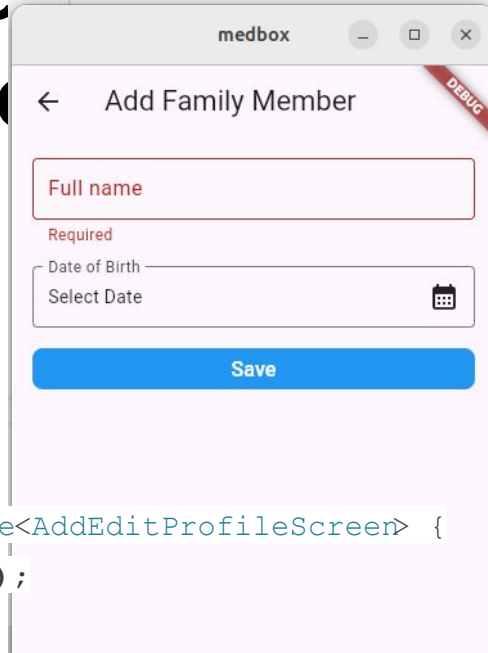
- On Screen
 - Using Form Validation techs

```
class _AddRecordScreenState extends State<AddEditProfileScreen> {  
  final _formKey = GlobalKey<FormState>();
```

```
  child: Form(  

```

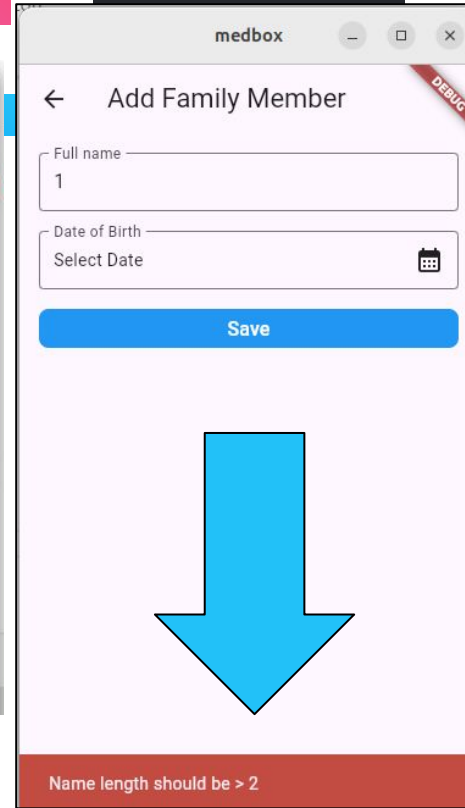
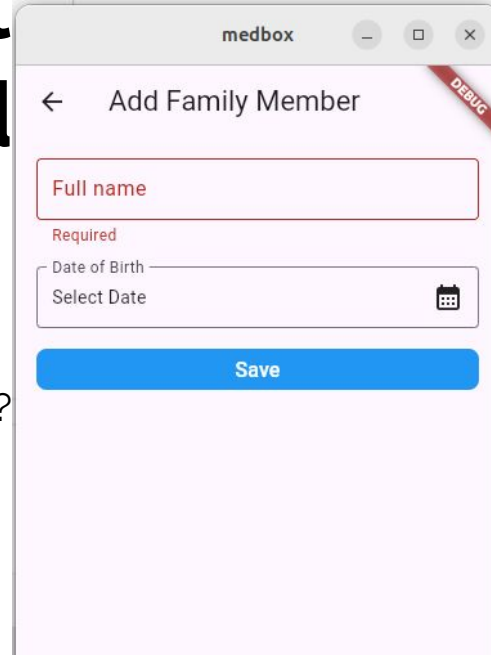
```
    validator: (v) => (v == null || v.isEmpty) ? "Required" : null,
```



Case Study for developing a Mobile app : Medbox

- **Validation of Data**

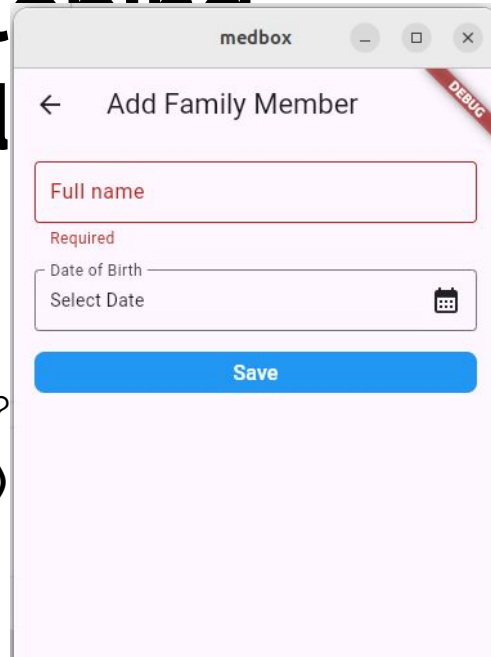
- + On Data Repo Layer
- How to show the Snackbar Message ?



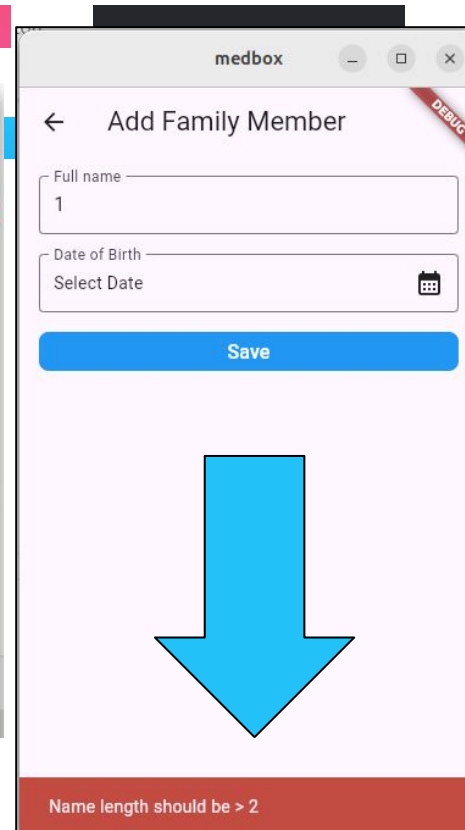
Case Study for developing a Mobile app : Medbox

- **Validation of Data**

- + On Data Repo Layer
- How to show the Snackbar Message ?
 - **Using BlocConsumer (Listener)**



The image shows a mobile app interface for 'medbox' with the title 'Add Family Member'. It features two input fields: 'Full name' and 'Date of Birth'. The 'Full name' field has a red border and the text 'Full name' in red, indicating a validation error. Below it, the word 'Required' is written in red. The 'Date of Birth' field has a calendar icon and the text 'Select Date'. A blue 'Save' button is at the bottom. A red 'DEBUG' banner is visible in the top right corner.

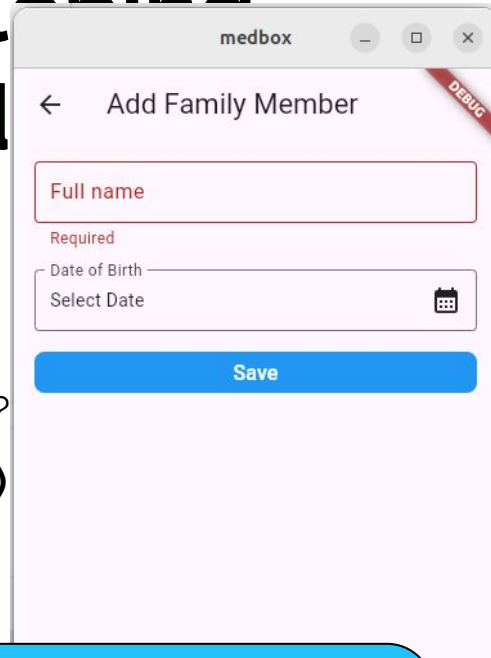


The image shows the same 'medbox' app interface, but with a large blue arrow pointing downwards in the center. At the bottom, a red banner displays the error message 'Name length should be > 2'. The 'Full name' field now contains the value '1'. The 'Date of Birth' field remains empty with the text 'Select Date'. The blue 'Save' button is still present. A red 'DEBUG' banner is visible in the top right corner.

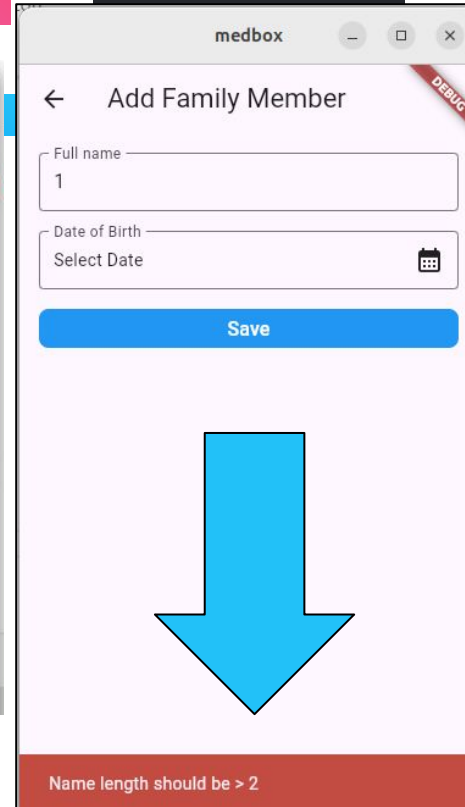
Case Study for developing a Mobile app : Medbox

- **Validation of Data**

- + On Data Repo Layer
- How to show the SnackBar Message ?
 - **Using BlocConsumer (Listener)**



The screenshot shows the 'Add Family Member' screen in the Medbox app. The 'Full name' field contains the text '1' and is highlighted with a red border and the text 'Required' below it. The 'Date of Birth' field is empty with a calendar icon. A blue 'Save' button is at the bottom. A red 'DEBUG' banner is visible in the top right corner.

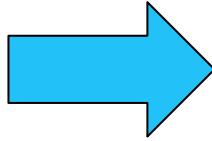


The screenshot shows the 'Add Family Member' screen in the Medbox app. The 'Full name' field contains the text '1' and is highlighted with a red border. The 'Date of Birth' field is empty with a calendar icon. A blue 'Save' button is at the bottom. A large blue arrow points downwards. A red banner at the bottom contains the text 'Name length should be > 2'. A red 'DEBUG' banner is visible in the top right corner.

But ? The Cubit/BloC is not used in this Screen ?

Do we need a Business Logic Component for adding ?

Tool for developing Flutter app : MedBox



medbox

← Add Family Member

Full name

Required

Date of Birth

Select Date

Save

DEBUG

medbox

← Add Family Member

Full name

Houdhaifa

Date of Birth

2025-11-12

Progress bar

DEBUG

Progress bar inside the button + button will become disabled to avoid double data entry

medbox

← Add Family Member

Full name
1

Date of Birth
Select Date

Save

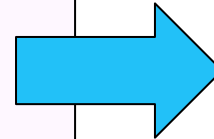
Name length should be > 2

medbox

← Add Family Member

Full name
Houdhaifa

Date of Birth
2025-11-12



medbox

Profiles

Imed
dob: 2025-11-11

Asma
dob: 2023-11-01

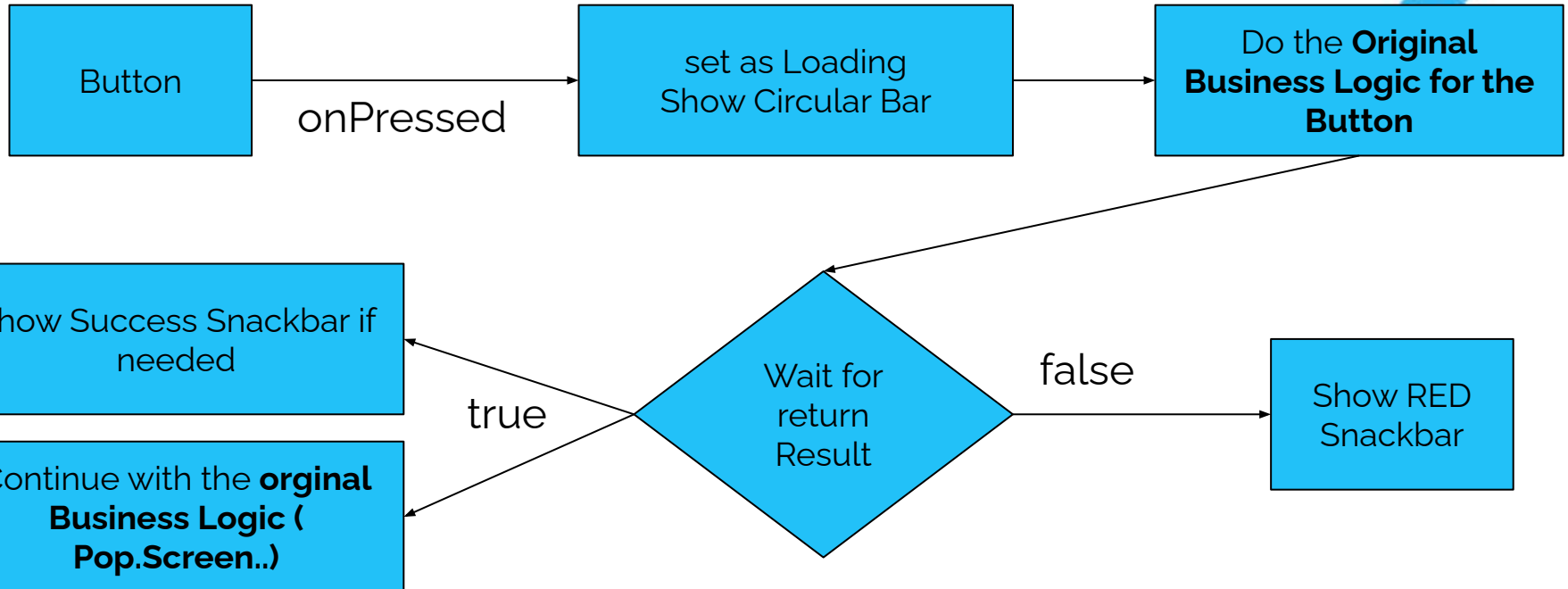
Houdhaifa
dob: 2025-11-12

+

Record is inserted successfully

Some Checks are done at the Repo Layer , the UI needs to show a snackbar message for this ?

Case Study for developing a Mobile app : MedBox



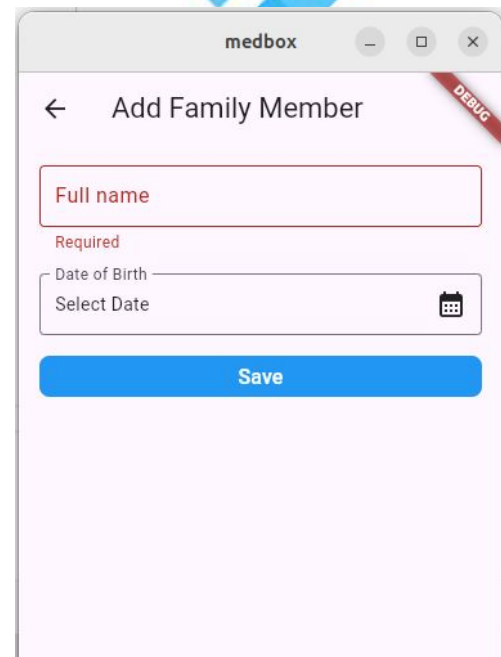
profile_add_edit.dart

```
class AddEditProfileScreen extends StatefulWidget {
  Map? instanceData;
  AddEditProfileScreen({super.key, this.instanceData});
  @override
  State<AddEditProfileScreen> createState() => _AddRecordScreenState();
}

class _AddRecordScreenState extends State<AddEditProfileScreen> {
  final _formKey = GlobalKey<FormState>();
  final TextEditingController titleCtrl = TextEditingController();
  DateTime? selectedDate;

  @override
  void initState() {
    super.initState();
    if (widget.instanceData != null) {
      titleCtrl.text = widget.instanceData!['name'];
      selectedDate = DateTime.parse(widget.instanceData!['dob']);
    }
  }

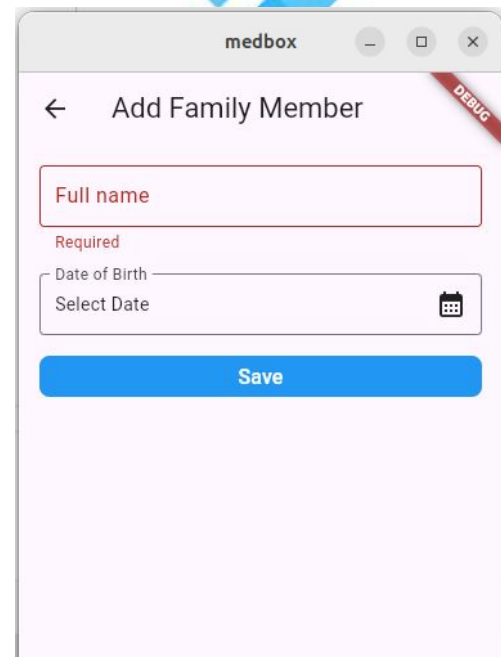
  @override
  Widget build(BuildContext context) {
    return Scaffold(
```



The screenshot shows a mobile application window titled "medbox" with a "DEBUG" banner in the top right corner. The screen displays a form titled "Add Family Member" with a back arrow on the left. The form contains two input fields: "Full name" and "Date of Birth". The "Full name" field is marked as "Required". The "Date of Birth" field has a calendar icon and the text "Select Date". At the bottom of the form is a blue "Save" button.

profile_add_edit.dart

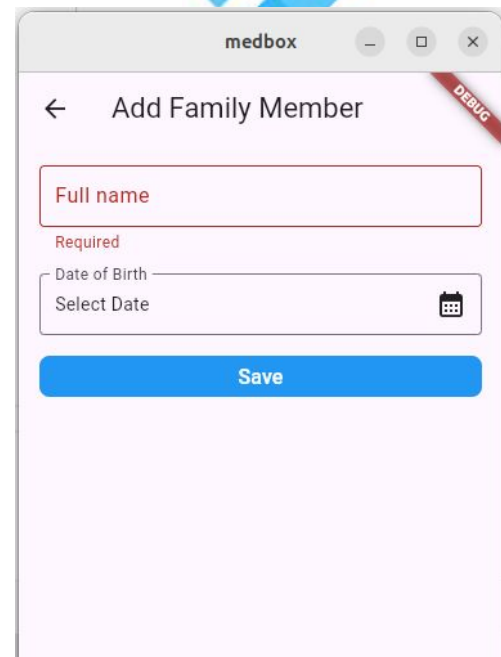
```
@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: Text(
        widget.instanceData == null
          ? "Add Family Member"
          : "Edit Family Member",
      ),
    ),
    body: Padding(
      padding: const EdgeInsets.all(16),
      child: Form(
        key: _formKey,
        child: Column(
          children: [
            // Title
            TextFormField(
              controller: titleCtrl,
              decoration: const InputDecoration(
                labelText: "Full name",
                border: OutlineInputBorder(),
              ),
              validator: (v) => (v == null || v.isEmpty) ? "Required" : null,
            ),
            const SizedBox(height: 16),
            // Date Picker
            InputDecorator(
              decoration: const InputDecoration(
                labelText: "Date of Birth",
                border: OutlineInputBorder(),
              ),
              child: InkWell(
                onTap: () async {
                  final date = await _showDatePicker(
                    context: context,
                    initialDate: widget.instanceData?.dateOfBirth ?? DateTime.now(),
                    firstDate: DateTime(2000),
                    lastDate: DateTime(2030),
                  );
                  if (date != null) {
                    titleCtrl.text = date.toIso8601String().split('T')[0];
                  }
                },
              ),
            ),
          ],
        ),
      ),
    ),
  );
}
```



profile_add_edit.dart

```
// Submit Button
Row(
  children: [
    Expanded(
      child: MyProgressButton(
        label: "Save",
        onPressed: handleSaveButton,
      ),
    ),
  ],
),
```

```
if (widget.instanceData != null) SizedBox(width: 10),
if (widget.instanceData != null)
  SizedBox(
    width: 100,
    child: ElevatedButton(
      onPressed:
        () => showDialog(
          context: context,
          builder:
            (context) => MyConfirmationDialog(
              dialogType: 'delete',
              title: "Really want to delete ?",
              message: "If you delete .",
              onConfirm: () async {
                var result = await context
                  .read<ProfilesCubit>()
                  .deleteItem(
```



medbox

← Add Family Member

Full name

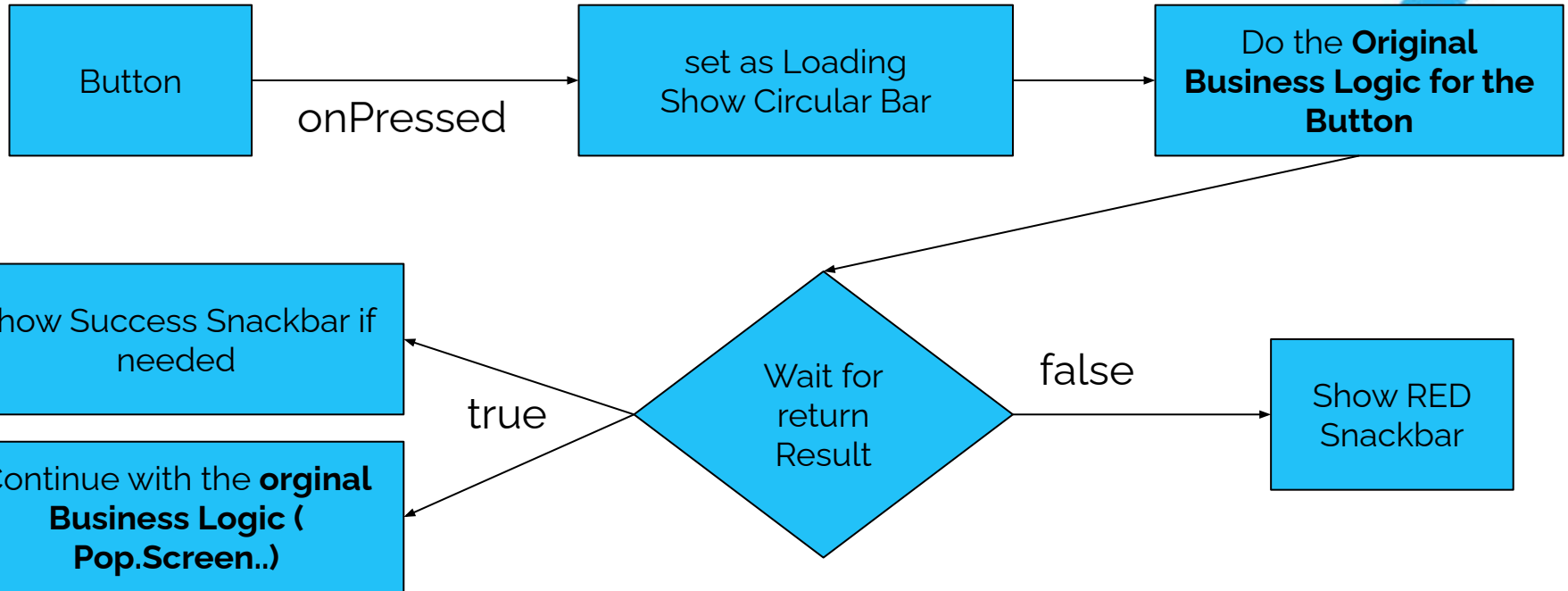
Required

Date of Birth

Select Date

Save

Case Study for developing a Mobile app : MedBox



profile_add_edit.dart

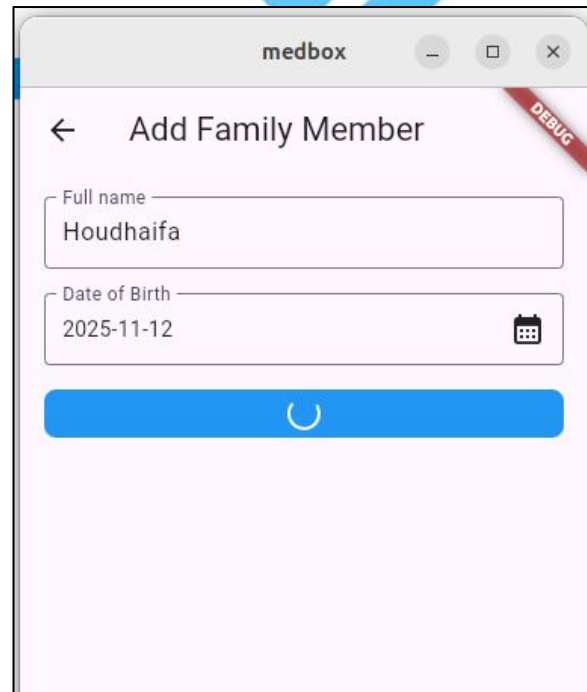
```
Future<ReturnResult> handleSaveButton() async {

  var result = ReturnResult(state: false, message: '');
  if (!_formKey.currentState!.validate()) return result;

  Map<String, dynamic> payload = {
    "name": titleCtrl.text,
    "dob": (selectedDate ?? DateTime.now()).toString().split(" ").first,
  };

  if (widget.instanceData == null) {
    result = await context.read<ProfilesCubit>().insertItem(payload);
  } else {
    Map<String, dynamic> new_payload = {
      ...widget.instanceData ?? {},
      ...payload,
    };
    result = await context.read<ProfilesCubit>().updateItem(new_payload);
  }
  if (result.state && Navigator.of(context).canPop()) {
    Navigator.of(context).pop();
  }
  return result;
}
```

Do the **Original**
Business Logic for the
Button



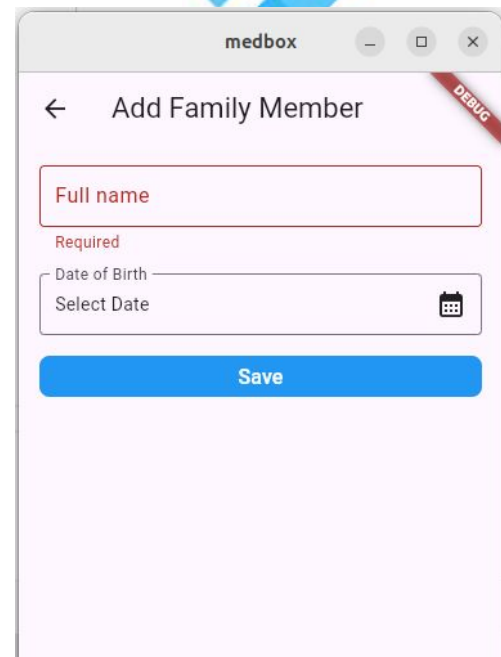
The screenshot shows a mobile application window titled "medbox" with a "DEBUG" banner in the top right corner. The screen displays a form titled "Add Family Member". The form has two input fields: "Full name" with the text "Houdhaifa" and "Date of Birth" with the date "2025-11-12" and a calendar icon. Below the form is a large blue button with a white circular arrow icon, indicating a loading or processing state.

profile_add_edit.dart

```
// Submit Button
```

```
Row(  
  children: [  
    Expanded(  
      child: MyProgressButton(  
        label: "Save",  
        onPressed: handleSaveButton,  
      ),  
    ],  
  ),
```

```
if (widget.instanceData != null) SizedBox(width: 10),  
if (widget.instanceData != null)  
  SizedBox(  
    width: 100,  
    child: ElevatedButton(  
      onPressed:  
        () => showDialog(  
          context: context,  
          builder:  
            (context) => MyConfirmationDialog(  
              dialogType: 'delete',  
              title: "Really want to delete ?",  
              message: "If you delete .",  
              onConfirm: () async {  
                var result = await context  
                  .read<ProfilesCubit>()  
                  .deleteItem(  
                    widget.instanceData!.id,
```



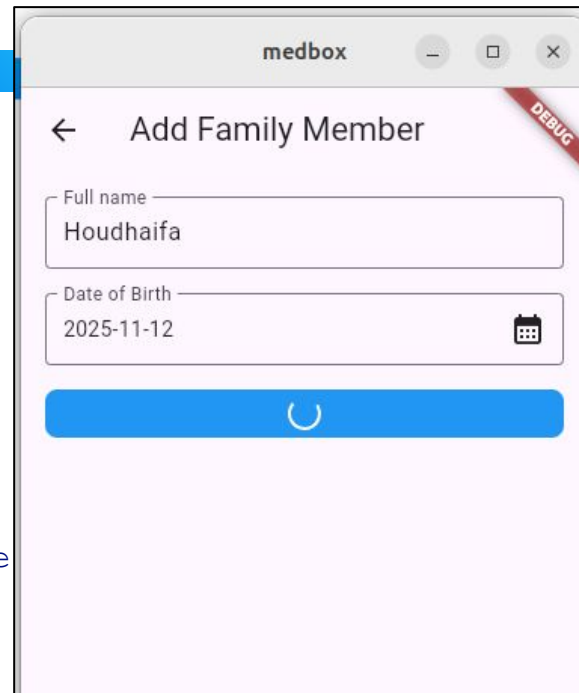
The screenshot shows a mobile application window titled "medbox" with a "DEBUG" banner in the top right corner. The screen displays a form titled "Add Family Member" with a back arrow on the left. The form contains two input fields: "Full name" and "Date of Birth". The "Full name" field is marked as "Required" in red text. The "Date of Birth" field has a calendar icon and the text "Select Date". At the bottom of the form is a blue "Save" button.

```

class MyProgressButton extends StatefulWidget {
  final String label;
  final String buttonType;
  final Function() onPressed;
  const MyProgressButton({
    this.buttonType = 'action',
    required this.label,
    required this.onPressed,
    super.key,
  });
  @override
  State<MyProgressButton> createState () => _MyProgressButtonState ();
}
class _MyProgressButtonState extends State<MyProgressButton> {
  bool isLoading = false;
  @override
  Widget build(BuildContext context) {
    final Color accentColor =
      widget.buttonType == 'danger' ? Colors.red : Colors.blue
    return FilledButton (
      onPressed: handleProgressButtonPress,
      style: ElevatedButton.styleFrom(
        backgroundColor: accentColor,
        foregroundColor: Colors.white,
        padding: const EdgeInsets.symmetric(horizontal: 24, vertical: 12),
        shape: RoundedRectangleBorder (borderRadius: BorderRadius.circular(8)),
        elevation: 0,
      ),
    ),
    child:
      isLoading
        ? CircularProgressIndicator(color: Colors.white, constraints: BoxConstraints(minHeight: 20,

```

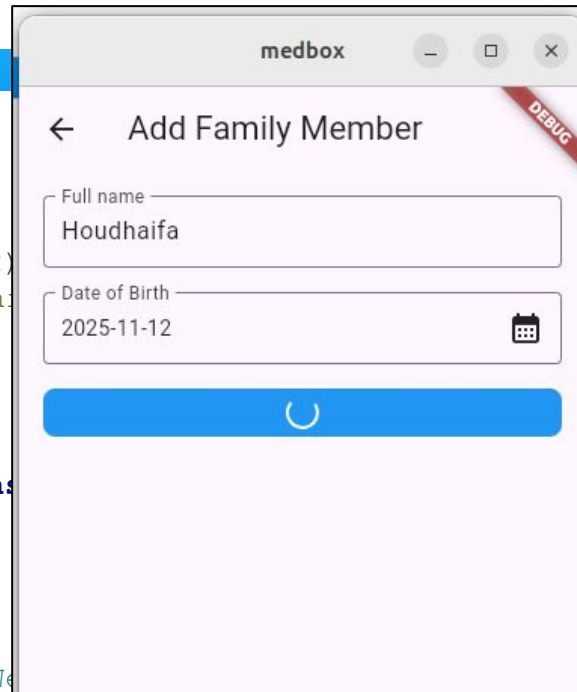
progress_button.dart



progress_button.dart

```
Widget build(BuildContext context) {  
  final Color accentColor =  
    widget.buttonType == 'danger' ? Colors.red : Colors.blue;  
  return FilledButton(  
    onPressed: handleProgressButtonPress,  
    style: ElevatedButton.styleFrom(  
      backgroundColor: accentColor,  
      foregroundColor: Colors.white,  
      padding: const EdgeInsets.symmetric(horizontal: 24, vertical: 12),  
      shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(12)),  
      elevation: 0,  
    ),  
    child:  
      isLoading  
        ? CircularProgressIndicator(color: Colors.white, const Size(24, 24))  
        : Text(widget.label,  
          style: const TextStyle(fontSize: 16, fontWeight: FontWeight.bold),  
        ),  
  );  
}
```

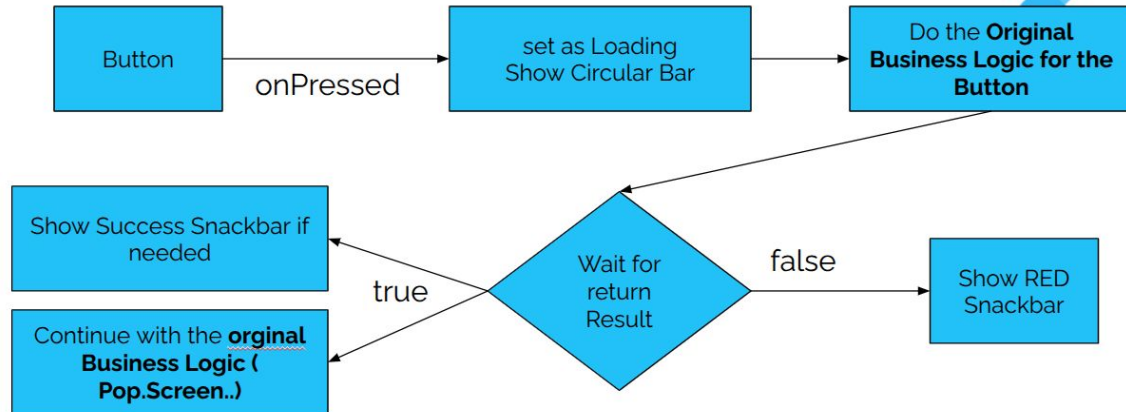
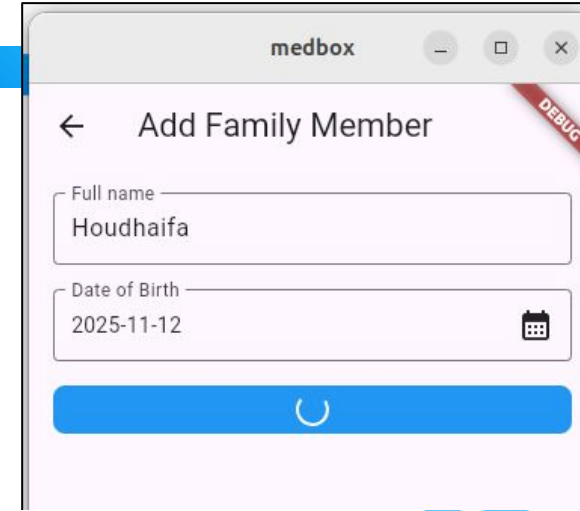
```
Future<bool> handleButtonPress () async {  
  if (!isLoading) {  
    try {  
      setState() {  
        isLoading = true;  
      };  
    }  
  }  
}
```



progress_button.dart

```
Future<bool> handleProgressButtonPress() async
  if (!isLoading) {
    try {
      setState(() {
        isLoading = true;
      });
      ReturnResult result = await widget.onPressed();

      if (widget.buttonType != 'silent' && result.message.length > 0)
        if (result.state)
          MySnackBar.success(context, result.message);
        else
          MySnackBar.error(context, result.message);
    } catch (e, trace) {
      print('$trace');
      MySnackBar.error(context, 'Operation Failed.. ${e.toString()}');
    }
    setState(() {
      isLoading = false;
    });
  }
  return true;
}
```



progress_button.d

```
Future<bool> handleProgressButtonPress() async {
  if (!isLoading) {
    try {
      setState(() {
        isLoading = true;
      });
      ReturnResult result = await widget.onPressed();

      if (widget.buttonType != 'silent' && result.message != null) {
        if (result.state)
          MySnackBar.success(context, result.message);
        else
          MySnackBar.error(context, result.message);
      } catch (e, trace) {
        print('$trace');
        MySnackBar.error(context, 'Operation Failed.. ${e}');
      }
      setState(() {
        isLoading = false;
      });
    }
    return true;
  }
}
```

Button

Show Success SnackBar
needed

Continue with the **org**
Business Logic (
Pop.Screen..)



```
Future<ReturnResult> handleSaveButton() async {
  var result = ReturnResult(state: false, message: null);
  if (!_formKey.currentState!.validate()) return result;

  Map<String, dynamic> payload = {
    "name": titleCtrl.text,
    "dob": (selectedDate ?? DateTime.now()).toString(),
  };

  if (widget.instanceData == null) {
    result = await context.read<ProfilesCubit>().init();
  } else {
    Map<String, dynamic> new_payload = {
      ...widget.instanceData ?? {},
      ...payload,
    };
    result = await context.read<ProfilesCubit>().update(new_payload);
  }

  if (result.state && Navigator.of(context).canPop()) {
    Navigator.of(context).pop();
  }

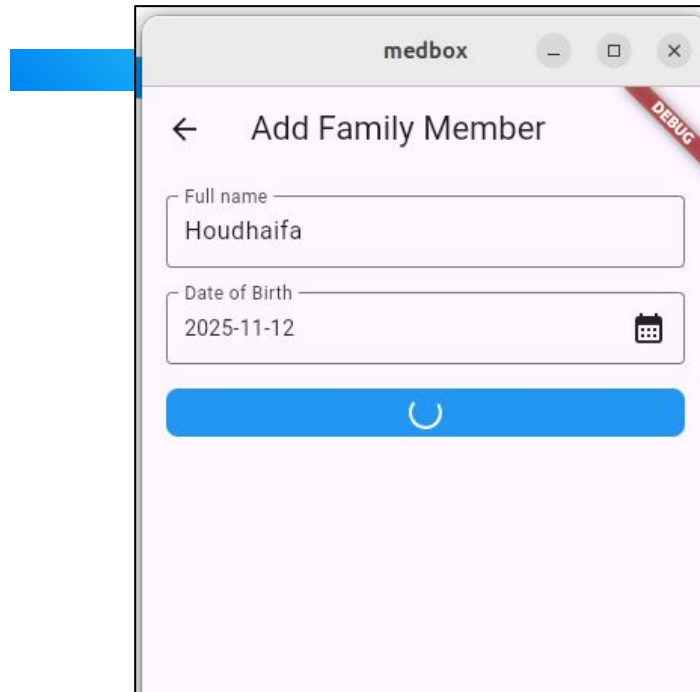
  return result;
}
```

```
class MySnackBar extends SnackBar {
  MySnackBar({required super.content});
```

```
static void success(
  BuildContext context,
  String message, {
  Duration duration = const Duration(seconds: 2),
  bool clearOtherSnakBeforShow = true,
}) {
  if (clearOtherSnakBeforShow) {
    ScaffoldMessenger.of(context).clearSnackBars();
  }
  ScaffoldMessenger.of(context).showSnackBar(
    SnackBar(
      backgroundColor: greenButtonColor,
      content: Text(message),
      duration: duration ?? Duration(seconds: 2),
    ),
  );
}

static void error(
  BuildContext context,
  String message, {
  Duration duration = const Duration(seconds: 6),
  bool clearOtherSnakBeforShow = true,
}) {
  if (clearOtherSnakBeforShow) {
```

my_snackbar.dart



medbox

Edit Family Member

Full name
ali

Date of Birth
2025-11-05

Save Delete

DEBUG

medbox

Edit Family Member

Full name
ali

Date of Birth
2025-11-05

Really want to delete ?

If you delete .

Cancel Delete

DEBUG

medbox

Edit Family Member

Full name
Houdhaifa

Date of Birth
2025-11-12

Really want to delete ?

If you delete .

Cancel

DEBUG

medbox

Profiles

lmed
dob: 2025-11-11

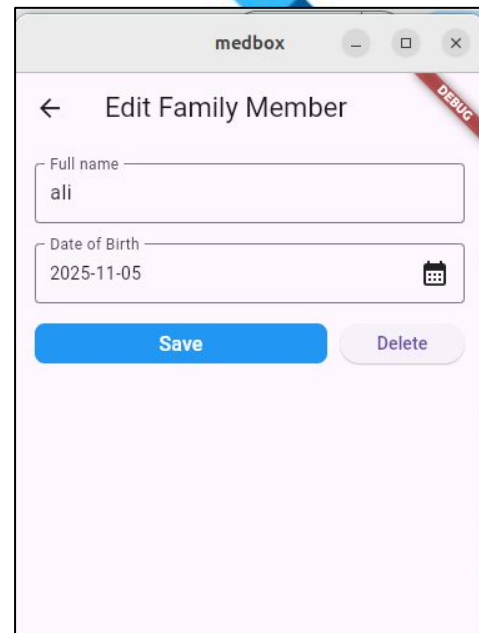
Record of id (2) is deleted successfully

+

DEBUG

profile_add_edit.dart

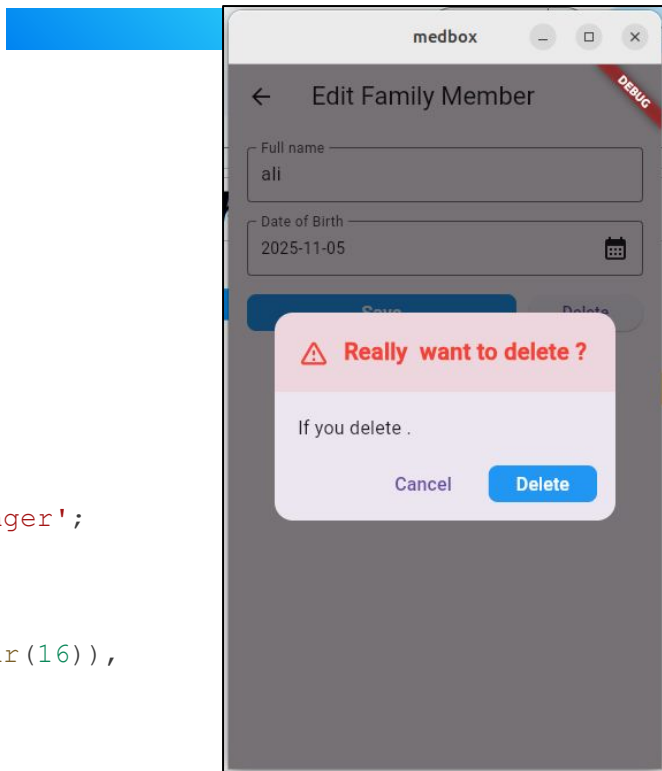
```
Row(
  children: [
    Expanded(
      child: MyProgressButton(
        label: "Save",
        onPressed: handleSaveButton,
      ),
    ),
    if (widget.instanceData != null) SizedBox(width: 10),
    if (widget.instanceData != null)
      SizedBox(
        width: 100,
        child: ElevatedButton(
          onPressed:
            () => showDialog(
              context: context,
              builder:
                (context) => MyConfirmationDialog(
                  dialogType: 'delete',
                  title: "Really want to delete ?",
                  message: "If you delete .",
                  onConfirm: () async {
                    var result = await context
                      .read<ProfilesCubit>()
                      .deleteItem(
                        widget.instanceData!['id'],
                      );
                    if (Navigator.of(context).canPop())
                      Navigator.of(context).pop();
                    return result;
                  },
                  onClose: () {},
                ),
            ),
        child: Text('Delete'),
      ),
  ],
)
```



The screenshot shows a mobile application window titled "medbox". The main screen is titled "Edit Family Member". It features two input fields: "Full name" with the value "ali" and "Date of Birth" with the value "2025-11-05". Below these fields are two buttons: "Save" (blue) and "Delete" (light blue). A red "DEBUG" banner is visible in the top right corner of the app window.

confirmation_dialog

```
class MyConfirmationDialog extends StatelessWidget {  
  final String title;  
  final String message;  
  final String dialogType;  
  final Function() onConfirm;  
  final Function() onClose;  
  
  const MyConfirmationDialog({  
    this.dialogType = 'confirm',  
    required this.title,  
    required this.message,  
    required this.onConfirm,  
    required this.onClose,  
    super.key,  
  });  
  
  @override  
  Widget build(BuildContext context) {  
    final bool isDanger = dialogType == 'delete' || dialogType == 'danger';  
    final Color accentColor = isDanger ? Colors.red : Colors.blue;  
  
    return Dialog(  
      shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(16)),  
      elevation: 8,  
      child: Column(  
        crossAxisAlignment: CrossAxisAlignment.stretch,  
        mainAxisAlignment: MainAxisAlignment.min,  
        children: [  
          // Title Bar with colored background  
          Container(  
            padding: const EdgeInsets.all(20)
```

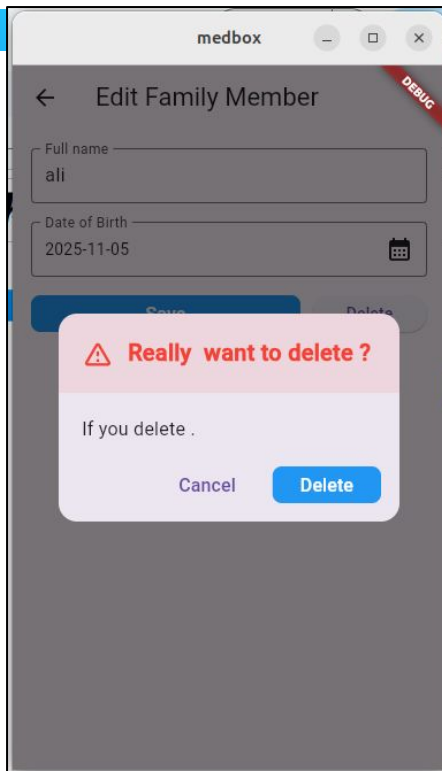


confirmation_dialog

```
// Action Buttons
Padding(
  padding: const EdgeInsets.fromLTRB(16, 0, 16, 16),
  child: Row(
    mainAxisAlignment: MainAxisAlignment.end,
    children: [
      // Close/Cancel Button
      TextButton(
        onPressed: () async {
          await onClose();
          if (Navigator.of(context).canPop()) {
            Navigator.of(context).pop();
          }
        },
        style: TextButton.styleFrom(
          padding: const EdgeInsets.symmetric(
            horizontal: 20,
            vertical: 12,
          ),
        ),
      ),
      child: const Text(
        'Close',
        style: TextStyle(fontSize: 16, fontWeight: FontWeight.w500),
      ),
    ],
  ),

  const SizedBox(width: 12),

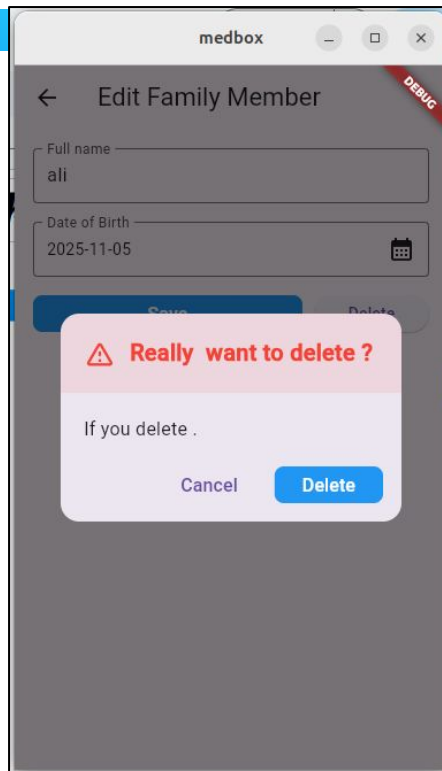
  // Confirm Button
  if (dialogType == 'confirm' || isDanger)
    MyProgressButton(
```



confirmation_dialog

```
const SizedBox(width: 12),

// Confirm Button
if (dialogType == 'confirm' || isDanger)
  MyProgressButton(
    onPressed: () async {
      var result = await onConfirm();
      if (Navigator.of(context).canPop()) {
        Navigator.of(context).pop();
      }
      return result;
    },
    label: 'Confirm',
  ),
],
),
),
],
),
);
}
}
```

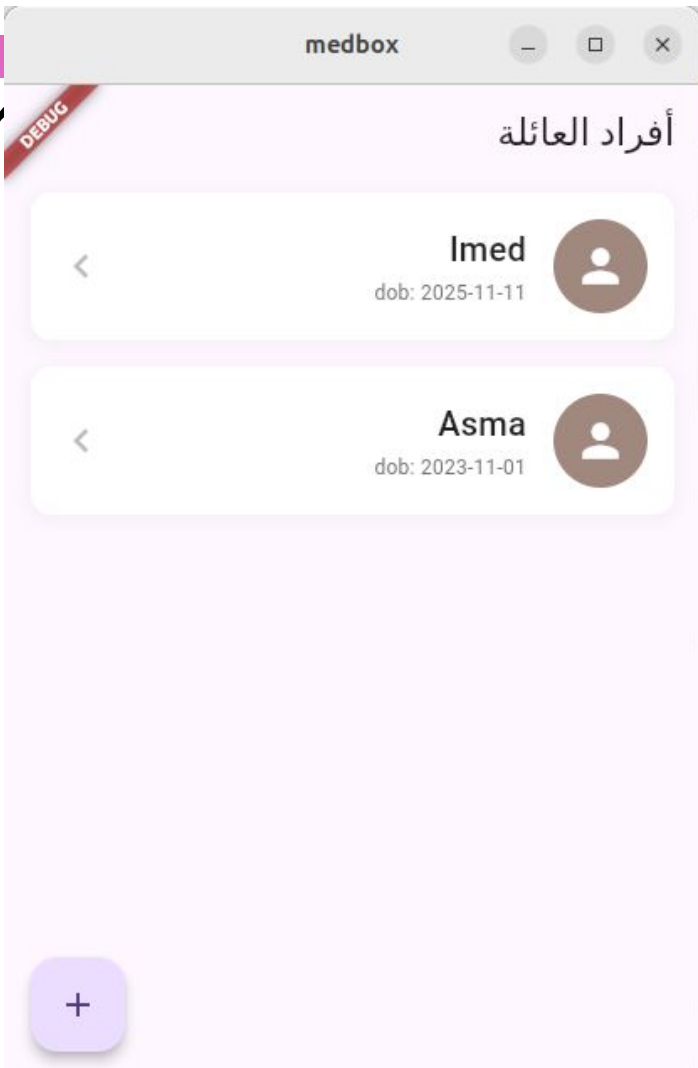




Localisation Different Languages

Localisation of your Flutter App.

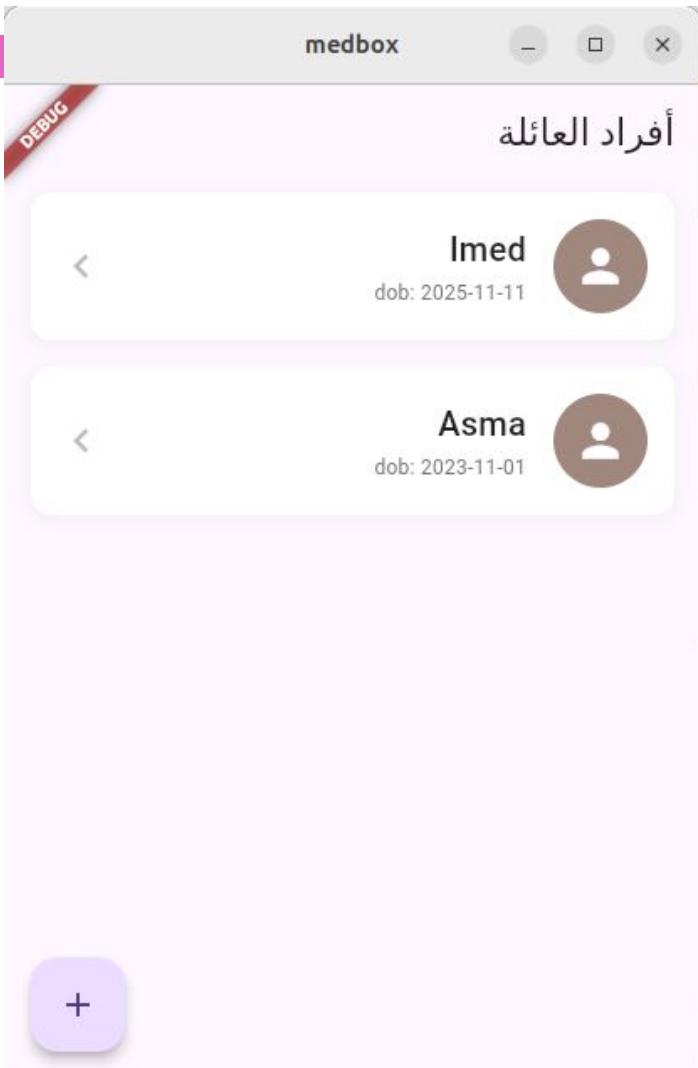
- Import the flutter Packages



Flutter App Localisation

- Import the flutter Packages + Generate: True

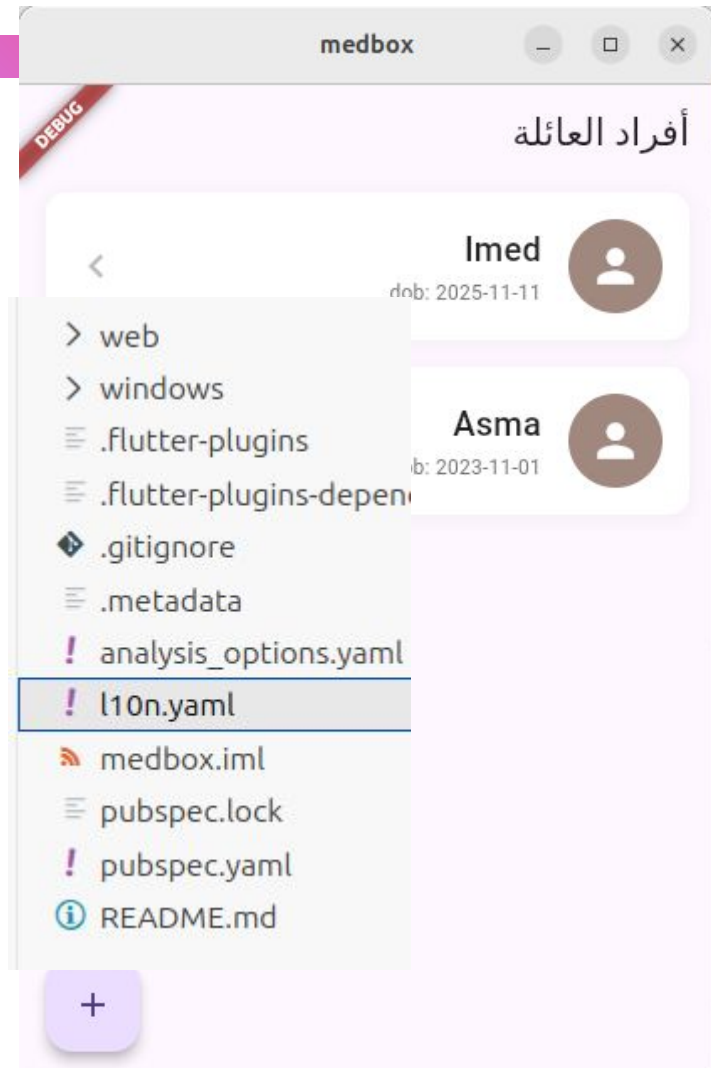
```
dependencies:  
  flutter:  
    sdk: flutter  
  flutter_localizations:  
    sdk: flutter  
  intl: any  
  
flutter_bloc: ^9.1.1  
get_it: ^9.1.0  
path: ^1.9.1  
sqflite: ^2.4.2  
sqflite_common_ffi: ^2.3.6  
  
dev_dependencies:  
  flutter_test:  
    sdk: flutter  
  flutter_lints: ^5.0.0  
  
flutter:  
  uses-material-design: true  
  generate: true
```



Flutter App Localisation

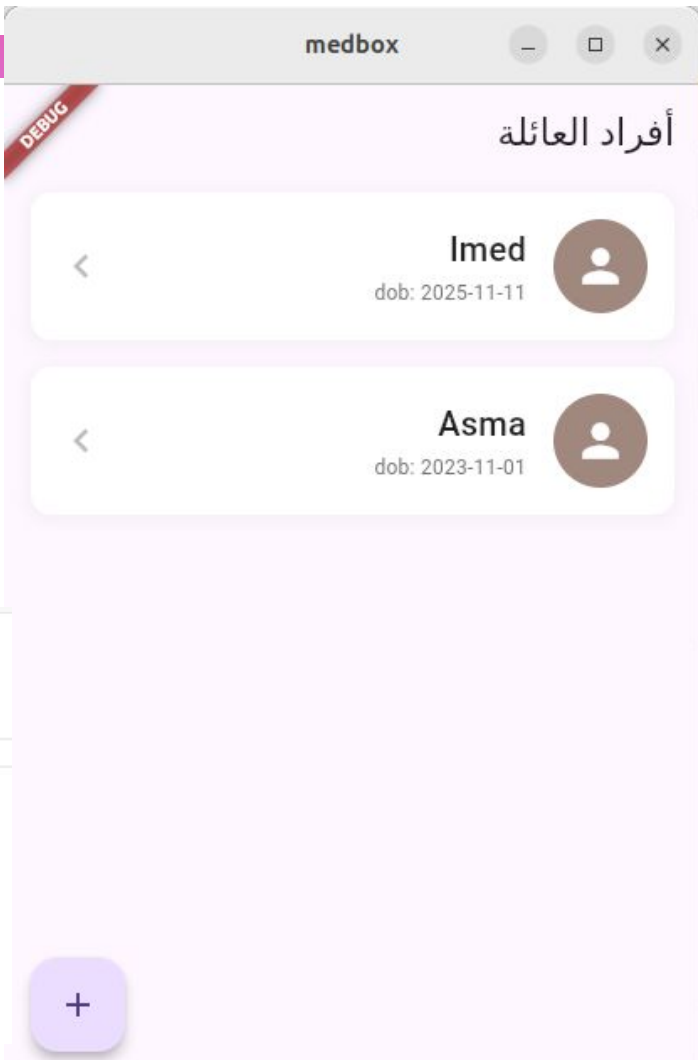
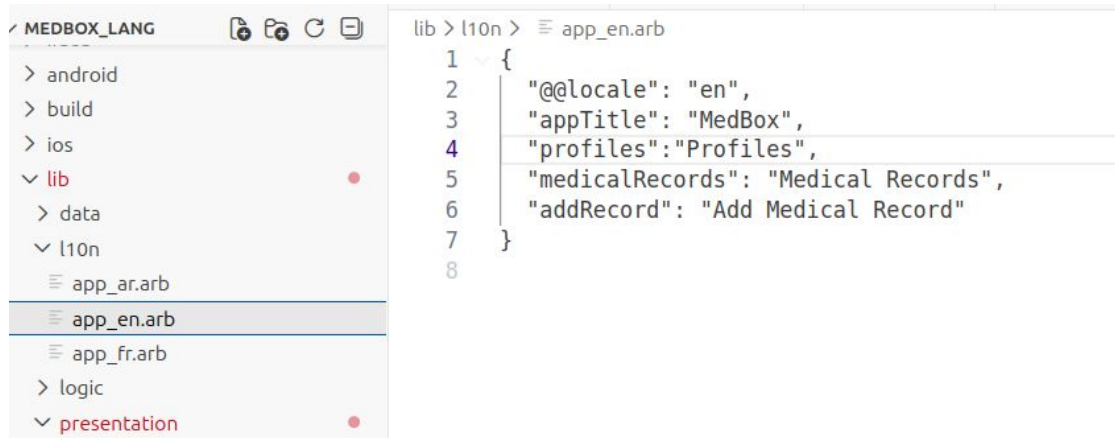
- Create the language Yaml file
 - Root folder of the project (NOT INSIDE LIB)
 - Create the file : l10n.yaml

```
arb-dir: lib/l10n
template-arb-file: app_en.arb
output-localization-file: app_localizations.dart
output-dir: lib/src/generated/l10n
```



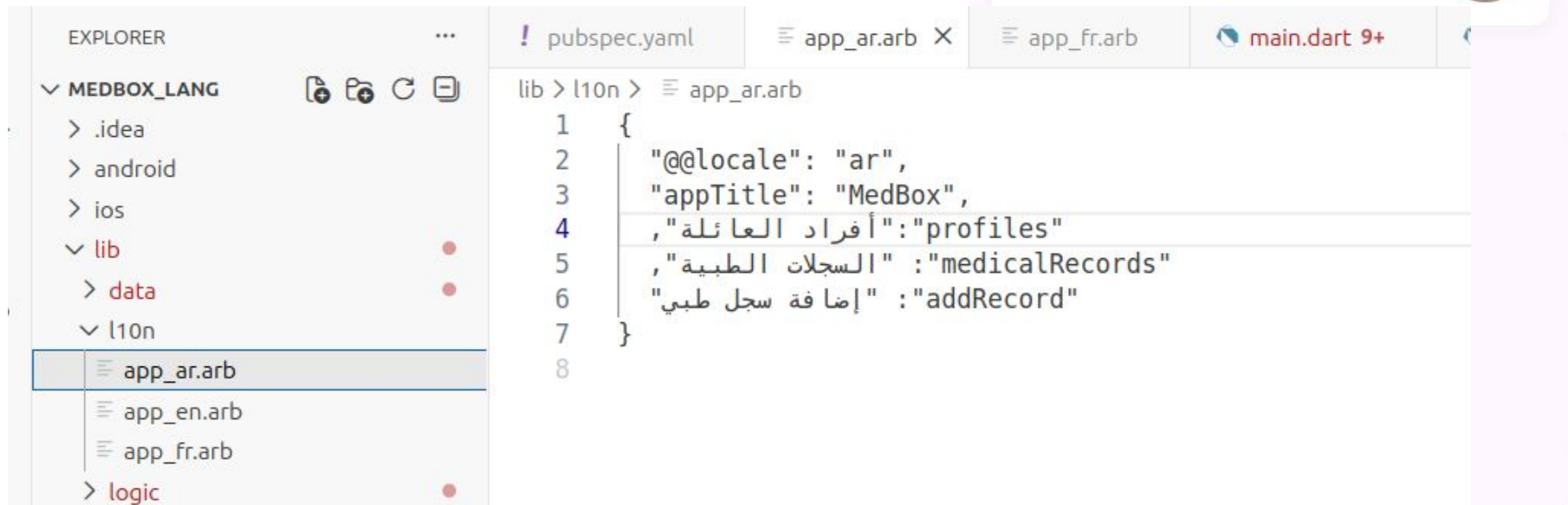
Flutter App Localisation

- Create the Translation arb Files
 - For each language, inside the lib/l10n, create the arb file



Flutter App Localisation

- Create the Translation arb Files
 - For each language, inside the lib/l10n, create the arb file



The screenshot displays the IDE environment for a Flutter application. The Explorer panel on the left shows the project structure, with the `lib/l10n` directory expanded, highlighting the `app_ar.arb` file. The main editor shows the content of `app_ar.arb`, which defines the Arabic localization for the app. The file content is as follows:

```
lib > l10n > app_ar.arb
1 {
2   "@@locale": "ar",
3   "appTitle": "MedBox",
4   "أفراد العائلة": "profiles"
5   , "السجلات الطبية": "medicalRecords"
6   , "إضافة سجل طبي": "addRecord"
7 }
8
```

On the right, a preview of the app shows the 'أفراد العائلة' (Family Members) screen. It displays two entries: 'Imed' with a date of birth of '2025-11-11' and 'Asma' with a date of birth of '2023-11-01'. Each entry has a back arrow and a profile icon. The app is running in debug mode, as indicated by the 'DEBUG' banner in the top left corner of the preview window.

```
import 'package:flutter_gen/gen_l10n/app_localizations.dart'
```

```
...
```

```
class MainApp extends StatelessWidget {  
  const MainApp({super.key});
```

```
  @override
```

```
  Widget build(BuildContext context) {
```

```
    return MultiBlocProvider(  
      providers: [BlocProvider(create: (_) => ProfilesCubit())],  
      child: const MaterialApp(  
        locale: Locale('ar'),  
        localizationsDelegates: [  
          AppLocalizations.delegate,  
          GlobalMaterialLocalizations.delegate,  
          GlobalWidgetsLocalizations.delegate,  
          GlobalCupertinoLocalizations.delegate,  
        ],  
        supportedLocales: [Locale('en'), Locale('ar'), Locale('ar')],  
        home: ProfileScreen(),  
      ),  
    );  
  }  
}
```

main.dart



Flutter App Localisation

- Create the Translation arb Files
 - For each language, inside the lib/l10n, create the arb file

```
import 'package:flutter_gen/gen_l10n/app_localizations.dart';
```

```
@override
```

```
Widget build(BuildContext context) {
```

```
  return Scaffold(  
    appBar: AppBar(title: Text(AppLocalizations.of(context)!.profiles)),  
    floatingActionButton: FloatingActionButton(  
      child: const Icon(Icons.add),  
      onPressed: () {
```



Flutter App Localisation

- Create the Translation arb Files

RTL is done automatically on the UI

```
import 'package:flutter_gen/gen_l10n/app_localizations.dart';

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(title: Text(AppLocalizations.of(context)!.profiles)),
    floatingActionButton: FloatingActionButton(
      child: const Icon(Icons.add),
      onPressed: () {
```





Scrolling..
Loading
More